DATABASE MANAGEMENT SYSTEMS

A Complete Study Material

In-depth explanations with a diagram for every concept

For GATE & Placement Preparation

Based on the GeeksforGeeks DBMS Tutorial structure

What's inside each chapter

Concept intuition (why it exists) • how it works step by step
• a diagram for every idea • worked examples • trade-offs and key insights • comparison tables

Table of Contents

Chapter 6: Normalization

6.1 The Problem: Anomalies from Redundancy
6.2 Functional Dependencies & Armstrong's Axioms
6.3 Attribute Closure
6.4 Normal Forms (1NF, 2NF, 3NF, BCNF, 4NF)
6.5 Lossless-Join and Dependency-Preserving Decomposition

Chapter 7: Transactions & Concurrency Control

7.1 What is a Transaction?
7.2 ACID Properties
7.3 Schedules and Serializability
7.4 Concurrency Control Protocols (Locking, 2PL, Timestamp)
7.5 Deadlocks in DBMS

Chapter 8: Recovery, File Organization & Indexing

8.1 Recovery — Surviving Crashes (WAL, Checkpoints)
8.2 File Organization (Heap, Sequential, Hash)
8.3 Indexing (Primary, Secondary, Clustered, Non-Clustered)
8.4 B-Trees and B+ Trees
8.5 Other Index Types (Bitmap, Inverted, Hash)

Appendix: Quick Reference Formulas & Key Points

Chapter 1

Basics of DBMS

Every business, app, and system depends on data. But raw data is useless if you can't reliably store, retrieve, and update it. This chapter explains what a DBMS is, why it exists, and how it's organized.

1.1 Database and DBMS

A **database** is an organized collection of related data. The data can be anything — customer names, product prices, sensor readings, blog posts — as long as it's structured in a way that makes storage and retrieval possible.

A **Database Management System (DBMS)** is software that manages this collection. It provides the tools to define the structure, store data efficiently, enforce integrity rules, control who can access what, run queries, and handle multiple users simultaneously — all without users worrying about how the bits are actually laid out on disk.

> ***Analogy:*** *Think of a database as a very organized filing cabinet, and the DBMS as an expert librarian. You don't reach into the cabinet yourself and rummage around; you tell the librarian what you need ("find all invoices from March 2025 over $10,000") and they bring it back. They also make sure two people don't grab the same folder at once, that nothing gets lost when the lights flicker, and that only authorized visitors can look at sensitive files.*

Common Examples

Relational DBMSes (RDBMS) store data in tables with fixed schemas. Examples: MySQL, PostgreSQL, Oracle Database, Microsoft SQL Server, SQLite. They use SQL as the query language and are the dominant type in most enterprises.

NoSQL DBMSes store data in more flexible ways: key-value pairs (Redis), document stores (MongoDB), wide-column stores (Cassandra), or graphs (Neo4j). They trade some relational features for scalability, flexibility, or performance on specific workloads.

This study material focuses on the relational model, which is what GATE and most placement interviews test in depth.

1.2 Why Not Just Use Files?

Before DBMSes became mainstream (roughly the 1970s), applications stored data directly in files — one file per kind of record, custom code to read and write. This works for small systems but breaks in predictable ways once you scale.

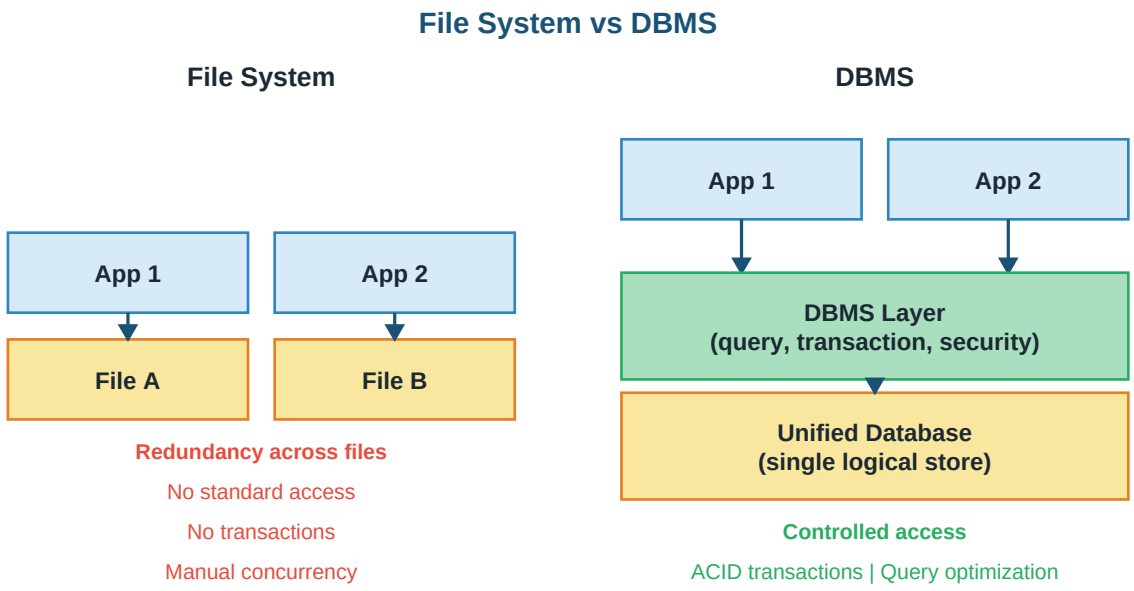


Figure: File-based storage vs a DBMS — problems and solutions

Problems with File-Based Systems

- **Data redundancy and inconsistency.** If a student's address is stored in the enrollment file, the fees file, and the transport file, updating it means finding and updating three copies. Miss one and now the data disagrees with itself.
- **Difficulty accessing data.** Want to run an ad-hoc query? Write a new program. There's no high-level query language — you're always at the byte level.
- **Data isolation.** Data lives in different file formats. Writing a report combining information from multiple files requires custom code to unify them.
- **Integrity problems.** Business rules (a fee must be positive, a course code must exist before students can enroll in it) must be enforced by every application separately. Forget it in one app and bad data sneaks in.
- **Atomicity problems.** When transferring money between accounts, both debit and credit must happen together. A crash between them leaves the system inconsistent.
- **Concurrent access anomalies.** Two clerks book the last airline seat at the same time — both succeed because both checked availability before either updated.
- **Security problems.** Every application must implement access control from scratch.

Key Insight: A DBMS provides one solution for all of these: centralized data, a high-level query language (SQL), integrity constraints, transactions with ACID properties, concurrency control, and access permissions.

1.3 DBMS Architecture

"DBMS architecture" is discussed in two different senses. One is how the DBMS is deployed relative to users (physical/tier architecture). The other is how the DBMS separates internal storage from user views (schema architecture).

Physical Architecture (1-Tier, 2-Tier, 3-Tier)

- **1-Tier:** DBMS and user are on the same machine. Simplest — used during development or in single-user tools like SQLite embedded in a phone app.
- **2-Tier (Client-Server):** Client applications on user machines connect directly to a database server. The client sends SQL; the server executes it and returns results. Older Windows office apps worked this way. Simpler than 3-tier but every client needs database credentials and drivers.
- **3-Tier:** A middle application server sits between clients and the database. Clients talk to the app server (over HTTP typically); the app server talks to the database. This is how modern web applications work.

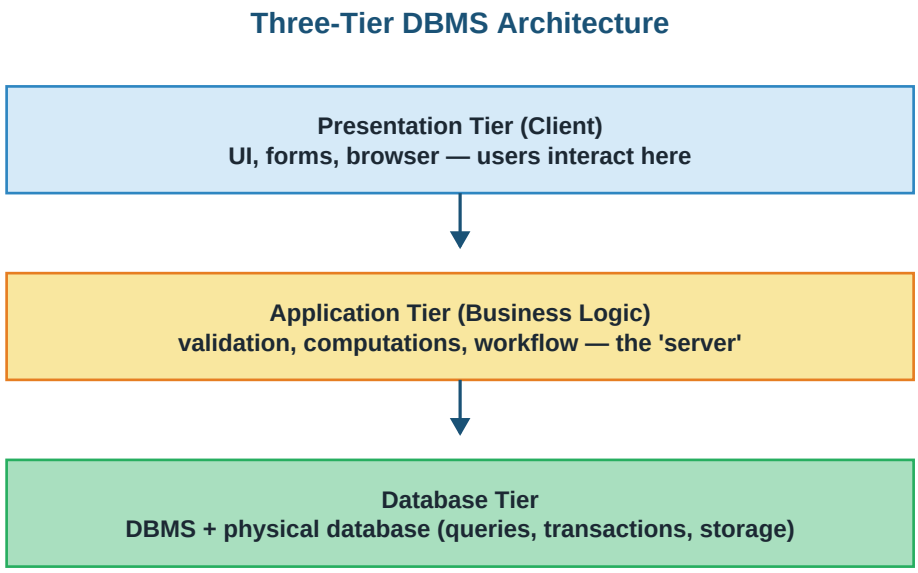


Figure: Three-tier architecture — separation lets each tier scale independently

Three-tier wins for large systems because each tier can scale independently. Need more concurrent users? Add more application servers. Need faster queries? Beef up the database. Clients only need a browser — no database drivers required.

Schema Architecture (Three-Schema / ANSI-SPARC)

A different notion of architecture, describing three levels of abstraction over the data. This is the one you're more likely to see in exam questions.

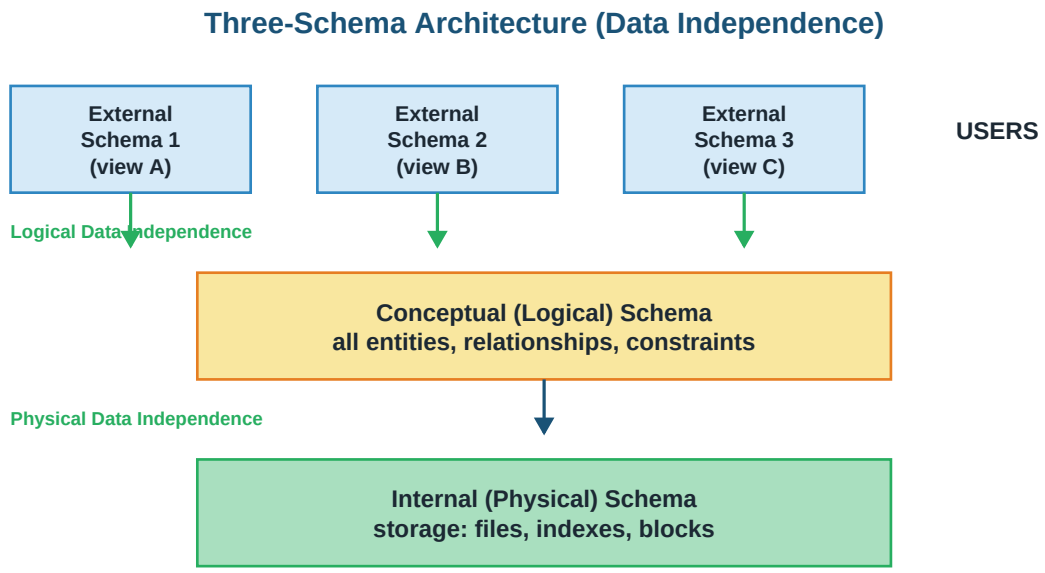


Figure: Three-schema architecture — enables data independence

- **Internal (Physical) Schema:** How data is physically stored — file organization, indexes, block layouts. Only DB implementers care about this level.
- **Conceptual (Logical) Schema:** The overall structure of the database — all entities, all attributes, all constraints — described logically, independent of storage. This is what a database administrator designs.
- **External Schemas (Views):** Each user or application sees a customized view of the data. A salesperson sees customer info; HR sees employee salaries; neither sees the other's data.

Data Independence

The three-schema architecture enables two crucial properties:

- **Physical Data Independence:** Change how data is physically stored (add an index, switch from heap to sorted file, move to different disk) without changing the conceptual schema or any application. Easy to achieve — almost every RDBMS provides this.
- **Logical Data Independence:** Change the conceptual schema (add a new attribute to a table, split a table into two) without changing existing external views or applications. Harder — requires that views can be redefined to still produce the same output.

1.4 Data Models

A **data model** is a way of describing data, its relationships, and its constraints. Different models make different assumptions.

- **Hierarchical model (1960s):** Data organized as a tree. Fast for parent-to-child queries; awkward for anything else. Used by IBM's IMS. Mostly historical now.
- **Network model (1970s):** Generalization allowing many-to-many relationships. More flexible than hierarchical but complex to navigate. Also mostly historical.
- **Relational model (Codd, 1970):** Data in tables (relations). Queries expressed in terms of set operations. Massive theoretical foundation. Dominant model today.

- **Object-oriented model:** Data as objects with methods. Fits programming languages nicely; less widely adopted than relational.
- **ER model:** Not for storage — for CONCEPTUAL DESIGN before you build. Chen introduced it in 1976. We use it in Chapter 2 to design databases before implementing them in the relational model.

1.5 DBMS Users

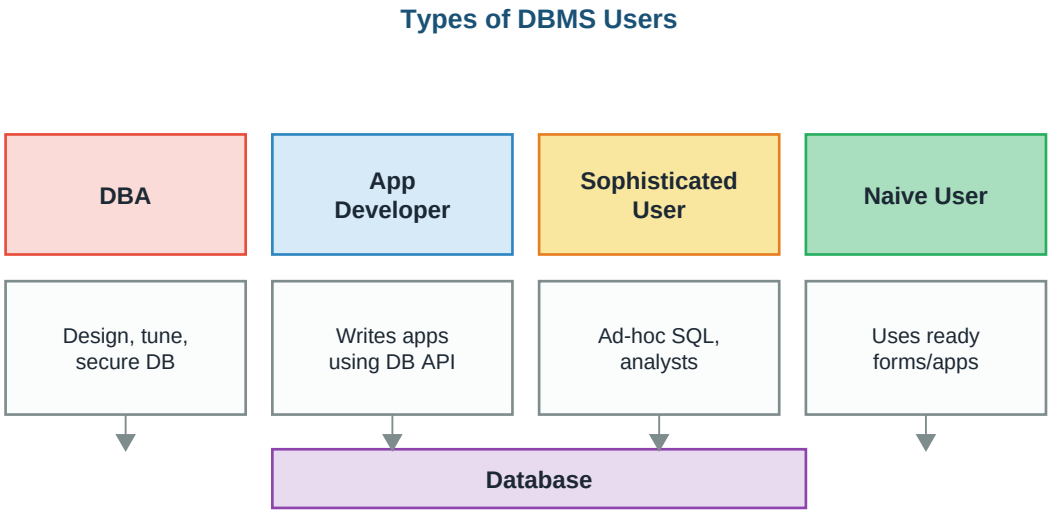


Figure: Categories of DBMS users

- **Database Administrator (DBA):** The most privileged user. Responsible for schema design, tuning, backup, security, user management, and disaster recovery. There are usually few DBAs, and they wield significant power.
- **Application Developers:** Write programs that use the database through APIs (JDBC, ORM libraries, etc.). They embed SQL in their code.
- **Sophisticated / Analytical Users:** Data analysts and business intelligence users who write their own ad-hoc queries directly. They know SQL but don't write applications.
- **Naive / End Users:** The vast majority. They interact with the database through pre-built forms, web pages, or apps — they don't know or care that a database is involved. Every time you order food online, you're a naive user of the restaurant's database.

1.6 Database Languages

Users interact with the database through structured languages. The most common set includes:

- **DDL (Data Definition Language):** Defines the schema. Creates, alters, and drops tables, indexes, views. In SQL: CREATE, ALTER, DROP, TRUNCATE.
- **DML (Data Manipulation Language):** Reads and modifies data. In SQL: SELECT, INSERT, UPDATE, DELETE.
- **DCL (Data Control Language):** Manages permissions. In SQL: GRANT, REVOKE.
- **TCL (Transaction Control Language):** Manages transaction boundaries. In SQL: COMMIT, ROLLBACK, SAVEPOINT.

SQL blends all four; the distinction is mostly conceptual. Some texts also split off DQL (Data Query Language) for SELECT, but usually SELECT is considered part of DML.

Chapter 2

Entity-Relationship Model

Before you build a database, you need to design it. The ER model gives you a visual, high-level language for capturing the real-world entities you care about and how they relate — without worrying about how they'll be stored.

2.1 What is the ER Model?

Peter Chen introduced the **Entity-Relationship (ER) Model** in 1976 as a way to describe database structure independently of implementation. The idea: talk about the real world in terms of **entities** (things that exist), **attributes** (properties of those things), and **relationships** (how those things connect). Then translate that description into whatever database technology you're using.

> ***Analogy:*** *Designing a database directly in tables is like drawing a house directly in bricks — you can do it, but you'll make expensive mistakes. An ER diagram is the blueprint. You draw it first, argue about it, revise it, and only then translate it into the tables that get built. Fixing a drawing is cheap; fixing a live database is not.*

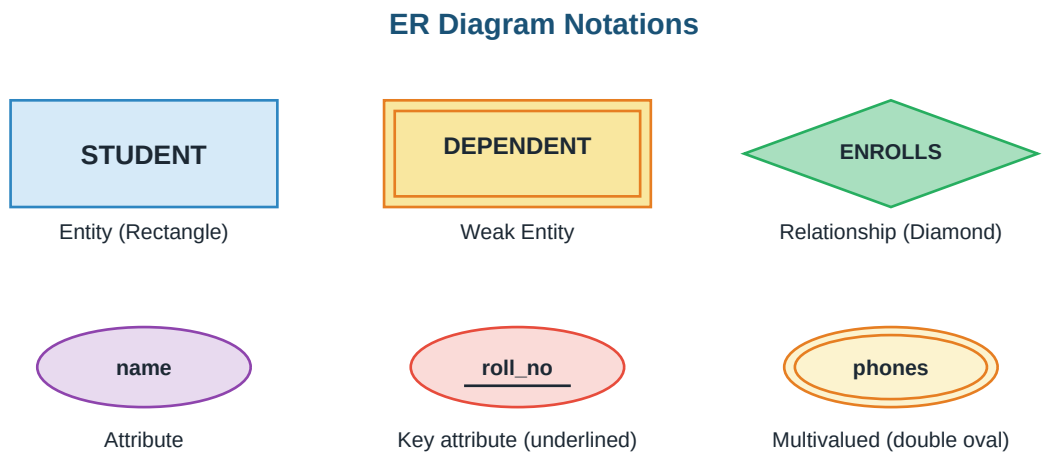


Figure: Standard ER diagram notation

2.2 Entities and Entity Sets

An **entity** is a real-world object or concept that can be distinctly identified — a student, a book, a bank account, a course. Each specific instance (student named Alice with ID 101) is an entity; the whole collection of similar entities is an **entity set** (all students).

In ER diagrams, entity sets are drawn as **rectangles** labeled with the entity name (usually in singular form and uppercase, e.g., STUDENT).

Strong vs Weak Entities

A **strong entity** can be uniquely identified by its own attributes (it has a primary key). Most entities are strong.

A **weak entity** cannot be uniquely identified by its own attributes alone — it depends on a related strong entity for identification. Drawn as a **double rectangle**, connected to the strong entity via an **identifying relationship** (double diamond).

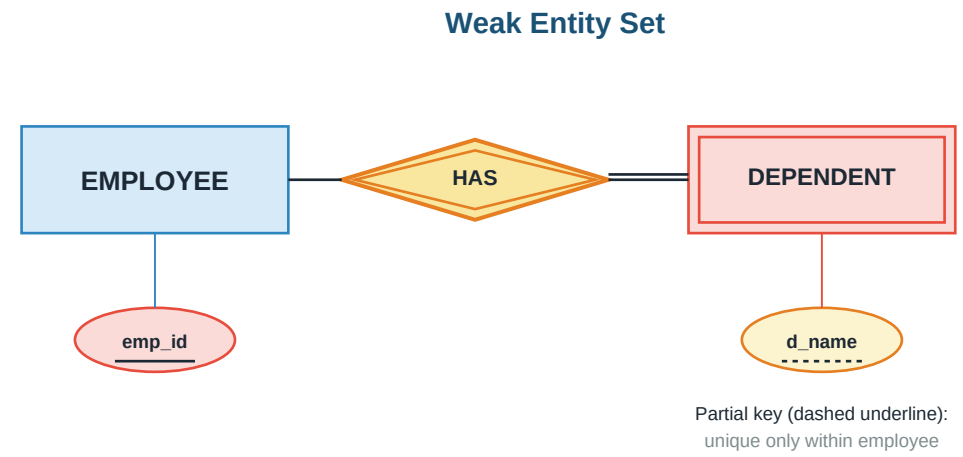


Figure: Weak entity — DEPENDENT is identified only via its EMPLOYEE parent

> **Example:** A dependent (spouse, child) of an employee cannot be uniquely identified by name alone — many employees might have dependents named "Sam". You need (emp_id, dependent_name) together. So DEPENDENT is weak, and d_name is a **partial key** (unique within one employee but not globally). The partial key gets a dashed underline in the diagram.

2.3 Attributes

An **attribute** is a property of an entity. Drawn as an **oval** connected to the entity rectangle. Attributes come in several flavors.

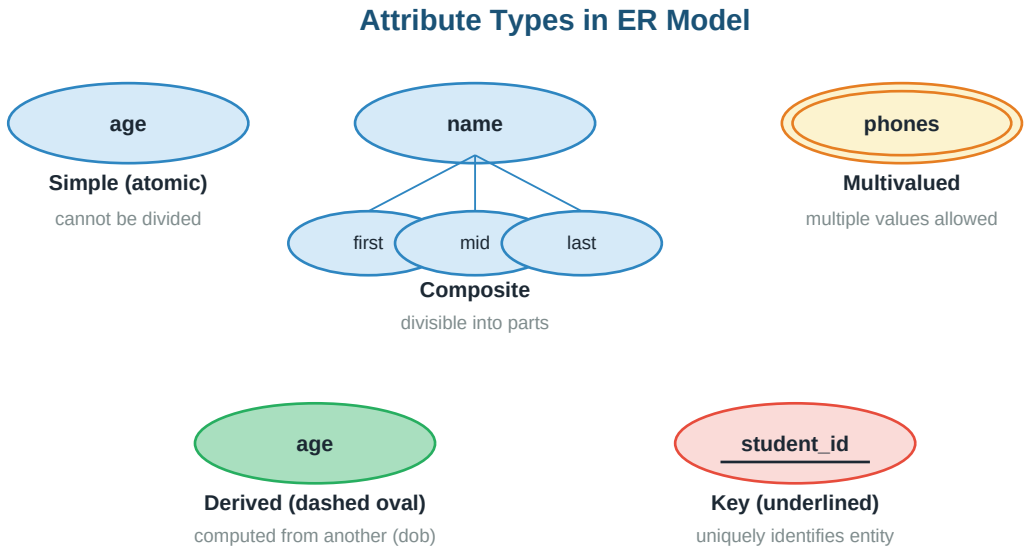


Figure: Attribute types with their ER notations

- **Simple (atomic):** Cannot be divided further. E.g., `age`, `sid`. Just a plain oval.
- **Composite:** Made of smaller parts. E.g., `name` may consist of first, middle, last. Drawn as an oval with smaller ovals branching off.
- **Multivalued:** Can hold multiple values for one entity. E.g., an employee may have multiple phone numbers. Drawn as a **double oval**.
- **Derived:** Computed from other attributes. E.g., `age` derived from `date_of_birth`. Drawn as a **dashed oval**.
- **Key attribute:** Uniquely identifies the entity (or is part of the key). Drawn as a plain oval with the attribute name **underlined**.

2.4 Relationships

A **relationship** is an association between two or more entities. Drawn as a **diamond** connecting the participating entities. Example: STUDENT is connected to COURSE via ENROLLS.

The **degree** of a relationship is the number of participating entity sets:

- **Unary (recursive):** Entity related to itself. E.g., EMPLOYEE 'supervises' EMPLOYEE.
- **Binary:** Two entities. Most common. E.g., STUDENT enrolls-in COURSE.
- **Ternary:** Three entities. E.g., DOCTOR treats PATIENT for DISEASE.

Cardinality (Mapping Ratios)

How many instances of one entity can be related to how many of the other? This is the **cardinality ratio**. Written near each side of the diamond in a diagram.

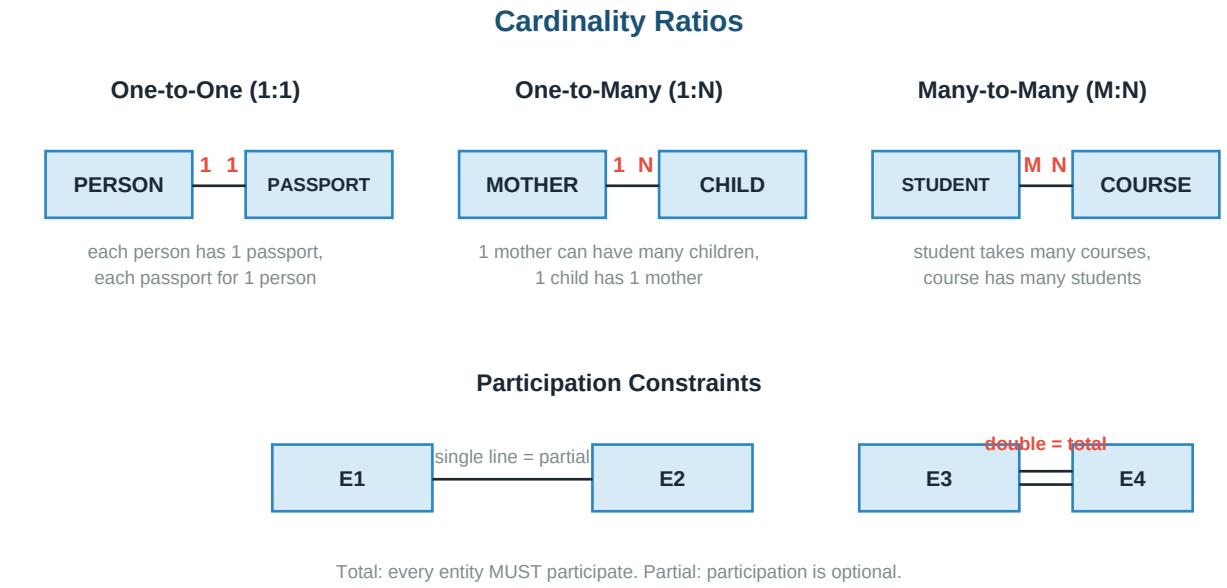


Figure: Cardinality ratios and participation constraints

- **One-to-One (1:1):** Each entity on one side relates to at most one on the other. E.g., PERSON and PASSPORT.

- **One-to-Many (1:N):** Each entity on one side may relate to many on the other; each on the other side to at most one on this side. E.g., MOTHER and CHILD.
- **Many-to-Many (M:N):** Any-to-any relations. E.g., STUDENT and COURSE.

Participation Constraints

How many entities on each side MUST participate?

- **Total participation:** Every entity of that type must participate in the relationship. Drawn with a **double line**. Example: every LOAN must have a BORROWER.
- **Partial participation:** Participation is optional — some entities may not be related. Drawn with a **single line**. Example: not every CUSTOMER has a LOAN.

2.5 Enhanced ER Concepts

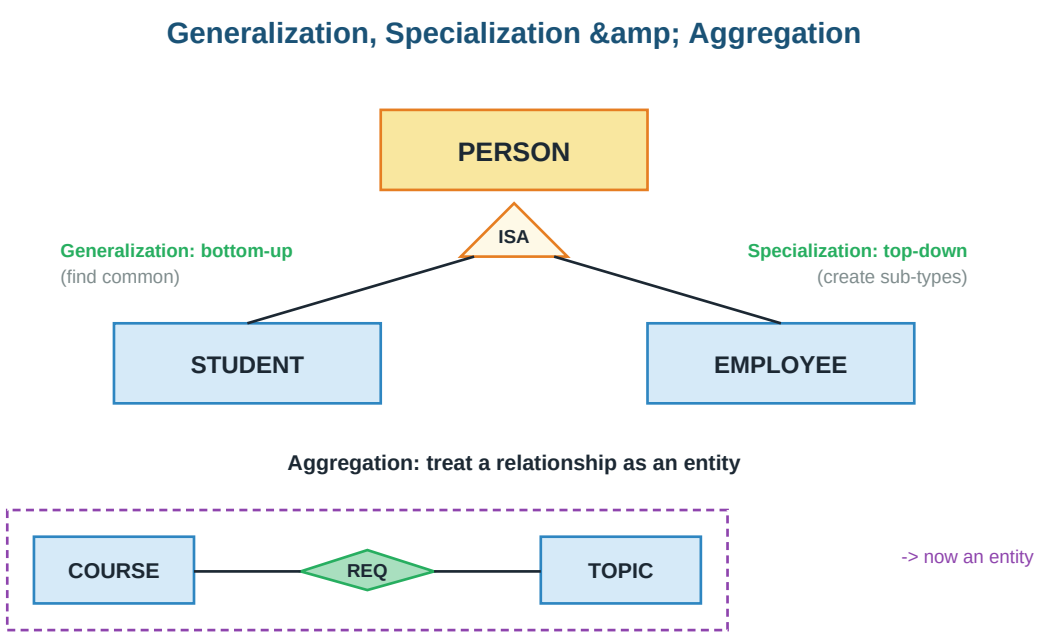


Figure: Generalization, specialization, and aggregation in ER

Generalization (Bottom-Up)

Take two or more similar entity sets and identify their common properties, producing a general parent entity set. Example: STUDENT and EMPLOYEE both have name, address, and phone — generalize into PERSON. Also called an **ISA** ("is-a") hierarchy, since student IS-A person.

Specialization (Top-Down)

The reverse: start with a general entity set and split it into more specific sub-types. Example: PERSON specializes into STUDENT and EMPLOYEE. Sub-types inherit parent attributes and add their own.

Generalization and specialization result in the same diagram — the difference is only the direction of thought.

Aggregation

Sometimes a relationship itself has properties, and needs to participate in another relationship. Standard ER doesn't allow relationships-between-relationships. **Aggregation** is the workaround: treat a whole relationship (entities + diamond) as a higher-level entity.

> **Example:** STUDENT enrolls in COURSE, and MANAGER approves that enrollment. We can't draw MANAGER-approves-(STUDENT-enrolls-COURSE) directly. Aggregation wraps the enrollment relationship in a box, and MANAGER connects to that box.

2.6 ER to Relational Mapping

Once your ER diagram is done, you convert it to relations (tables). This is straightforward if you follow the rules — much of database design is applying these mechanically.

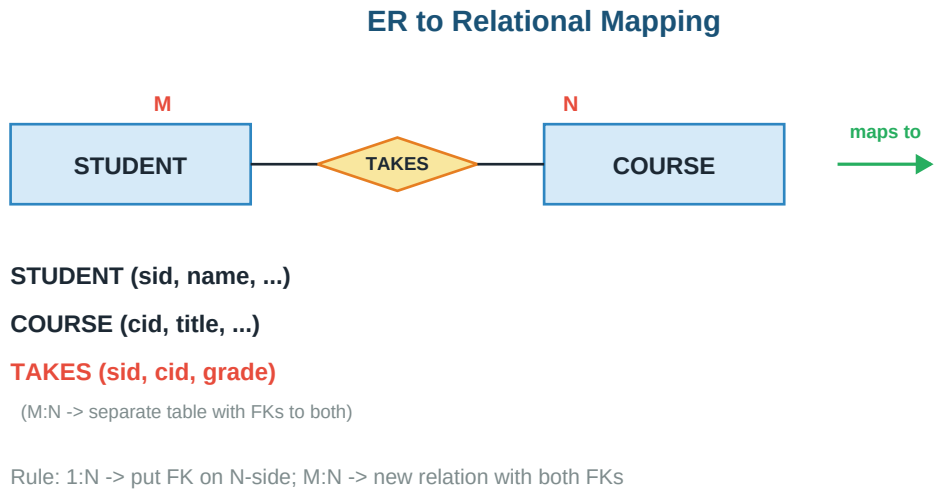


Figure: Turning an ER diagram into relations

- **Strong entity set:** Create one relation with all the entity's attributes; the entity's key becomes the relation's primary key.
- **Weak entity set:** Create a relation with the weak entity's attributes PLUS the primary key of its owner (strong) entity as a foreign key. The primary key is the composite of owner key + partial key.
- **1:1 relationship:** Add the primary key of one side as a foreign key in the other. Prefer the side with total participation (won't have NULLs).
- **1:N relationship:** Put the "1" side's primary key as a foreign key on the "N" side. No new table needed.
- **M:N relationship:** Create a NEW relation whose primary key is the combination of both sides' primary keys. Add relationship attributes if any. This is the only case where a relationship becomes its own table.
- **Multivalued attribute:** Create a separate table with the entity's key + the multivalued attribute. E.g., EMPLOYEE(emp_id, name) + EMPLOYEE_PHONE(emp_id, phone).
- **Composite attribute:** Flatten into its parts. Name (first, mid, last) becomes three columns: first_name, mid_name, last_name.
- **Generalization/Specialization:** Multiple options — one table for parent + separate tables for children, or one big table for all children with a type discriminator column, or one table per child (parent attributes duplicated). Choice depends on query patterns and how strict the sub-typing is.

Chapter 3

The Relational Model

Edgar Codd's 1970 relational model transformed databases. Almost every serious database today is built on it. This chapter covers its structure, constraints, and the powerful theory that underlies SQL.

3.1 Structure of the Relational Model

In the relational model, all data is organized as **relations** — which most of us call tables. But "table" is a bit informal; the mathematical term is precise, and understanding it clarifies a lot of confusion later.

Formal Terminology

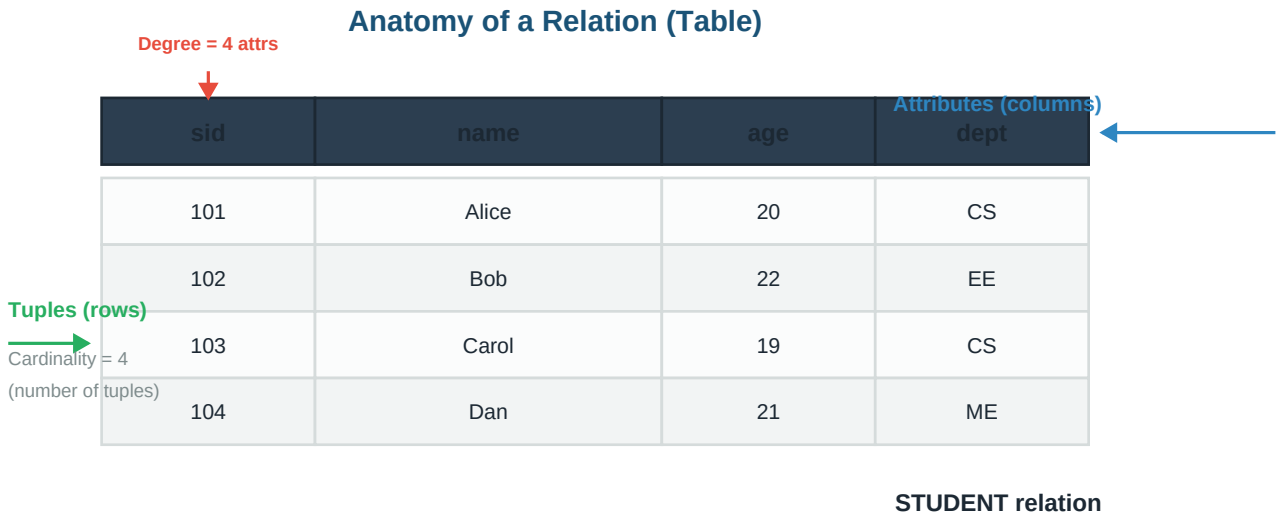


Figure: Anatomy of a relation — attributes, tuples, degree, cardinality

- **Relation:** A table. Formally, a set of tuples all sharing the same set of attributes.
- **Attribute:** A column of the relation. Each attribute has a name and a **domain** — the set of allowed values (e.g., age has domain "integers from 0 to 150").
- **Tuple:** A row of the relation. A tuple is a specific mapping from each attribute to a value in its domain.
- **Degree (arity):** Number of attributes in the relation. STUDENT with (sid, name, age, dept) has degree 4.
- **Cardinality:** Number of tuples. Changes as rows are inserted or deleted.
- **Schema:** The relation's structure — its name plus its attributes and their domains. Denoted as R(A1, A2, ..., An).
- **Instance:** The actual set of tuples in the relation at a given moment. The schema is fixed; the instance changes constantly.

Properties of Relations

Not everything that looks like a table is a valid relation. Codd required:

- **All values are atomic.** A cell contains a single value, not a list. This is the essence of First Normal Form (see Chapter 6).
- **Row order doesn't matter.** A relation is a SET of tuples. Whether Alice comes before Bob is not part of the relation.
- **Column order doesn't matter.** Attributes are identified by name, not position.
- **No duplicate tuples.** Every row is unique — because it's a set. (SQL relaxes this and calls them "multisets" or "bags", but the theoretical relational model doesn't allow duplicates.)

3.2 Keys

A **key** is a set of attributes whose values are unique across all tuples. Keys let us identify individual rows, enforce integrity, and connect relations to each other. There's a hierarchy of key concepts.

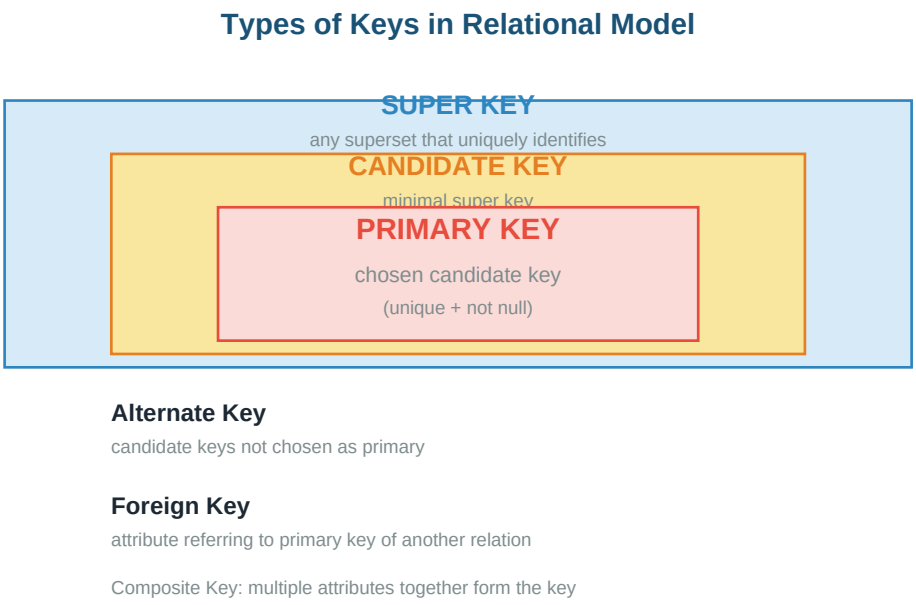


Figure: The nested hierarchy of keys — every primary key is a candidate key is a super key

Super Key

Any set of attributes that uniquely identifies a tuple. If STUDENT has attributes (sid, name, age, dept) and sid is unique, then **{sid}**, **{sid, name}**, **{sid, age, dept}**, and even **{sid, name, age, dept}** are all super keys. There are usually many super keys because any set containing a unique attribute is one.

Candidate Key

A **minimal** super key — one where no attribute can be removed and still have a super key. If both sid and email are individually unique, {sid} and {email} are both candidate keys. {sid, email} is a super key but not a candidate key (email alone suffices).

Primary Key

The one candidate key chosen to uniquely identify tuples. A relation can have multiple candidate keys, but exactly one primary key. Primary key values cannot be NULL. Typically stable, short, and simple — which is

why we invent surrogate IDs (student_id, order_number) rather than using natural keys like (name, birthdate).

Alternate Key

Candidate keys that weren't chosen as primary. If both sid and email are candidate keys and sid is primary, then email is an alternate key. Still enforced as unique.

Foreign Key

An attribute (or set of attributes) in one relation that refers to the primary key of another (or the same) relation. Foreign keys are how we link tables together.

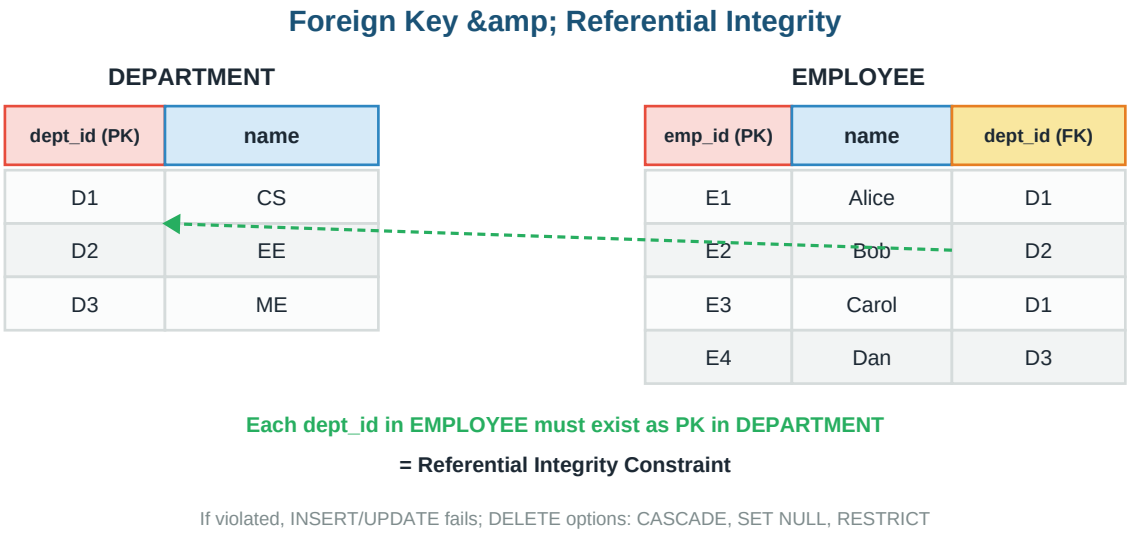


Figure: Foreign key — dept_id in EMPLOYEE references dept_id in DEPARTMENT

A foreign key value in EMPLOYEE must either be NULL or must match a real primary key value in DEPARTMENT. This is **referential integrity**: no orphan references.

Composite Key

A key made of two or more attributes together. Common in relationship tables. E.g., in the ENROLL table with (sid, cid, grade), neither sid nor cid alone is unique — but (sid, cid) is.

3.3 Integrity Constraints

Constraints are rules the database enforces on every insert, update, and delete. If an operation would violate a constraint, the DBMS rejects it.

- **Domain Constraint:** Each attribute value must be in its declared domain. Age must be an integer between 0 and 150. Salary must be non-negative. Character length limits count too.
- **Entity Integrity (Primary Key) Constraint:** Primary key values cannot be NULL, and cannot repeat within the relation.
- **Referential Integrity (Foreign Key) Constraint:** A foreign key value must either be NULL or match an existing primary key value in the referenced relation. Enforced on INSERT and UPDATE (of the child) and on DELETE and UPDATE (of the parent).
- **Key Constraint:** Every candidate key must have unique values across tuples.

- **General / User-Defined Constraints:** Business rules that don't fit the above categories. "Manager salary must be greater than employees they manage." Expressed via CHECK constraints or triggers.

What Happens When You Delete a Referenced Row?

If we delete department D1 while employees still refer to it, what should happen? SQL offers several options via ON DELETE clauses:

- **RESTRICT / NO ACTION:** Refuse the delete. Default in most databases.
- **CASCADE:** Delete the referring rows too. Deleting department D1 would delete all its employees.
- **SET NULL:** Set the foreign key to NULL in referring rows. Employees survive but their dept_id becomes NULL.
- **SET DEFAULT:** Set the foreign key to a default value.

Key Insight: Choose CASCADE only when the child is truly meaningless without the parent (like line items of an order — delete the order, line items must go too). For loosely coupled data, prefer RESTRICT.

Chapter 4

Relational Algebra & Calculus

SQL is what we write day to day. But underneath, databases use formal query languages to reason about and optimize queries. Relational algebra is procedural ("do this, then this"); relational calculus is declarative ("find rows satisfying this condition"). Both have equivalent expressive power.

4.1 What is Relational Algebra?

Relational algebra is a set of operations that take one or two relations as input and produce a relation as output. This **closure property** — output is always a relation — is what makes it composable. You can chain operations arbitrarily.

There are six **fundamental operators**. Everything else can be built from these:

- **Selection** (sigma) — pick rows
- **Projection** (pi) — pick columns
- **Union** (U) — combine two relations
- **Set Difference** (-) — rows in one but not the other
- **Cartesian Product** (X) — pair every tuple with every other
- **Rename** (rho) — rename relations or attributes

Derived operations built from these: Intersection, Join (natural, theta, equi, outer), Division.

4.2 Selection and Projection

Select (sigma) and Project (pi) Operations

Input: STUDENT

sid	name	age
101	Alice	20
102	Bob	22
103	Carol	19
104	Dan	21

sigma(age > 20)(STUDENT)

select rows where age > 20

sid	name	age
102	Bob	22
104	Dan	21

pi(name)(STUDENT)

project only 'name' column

name
Alice
Bob
Carol
Dan

pi(name, age)(sigma(age>20))

combine both

name	age
Bob	22
Dan	21

Figure: Selection filters rows; projection filters columns

Selection (sigma)

Selection extracts tuples that satisfy a given predicate. Denoted `sigma_condition(R)`. The output has the same schema as R but only the matching tuples.

```
sigma_{age > 20}(STUDENT)
-> tuples where age > 20

sigma_{dept='CS' AND age < 22}(STUDENT)
-> combine conditions with AND, OR, NOT
```

Projection (pi)

Projection keeps only specified attributes. Denoted `pi_attribute_list(R)`. The result has fewer columns; and being a set, duplicates are automatically removed.

```
pi_{name, dept}(STUDENT)
-> relation with only these two columns
-> duplicates removed (set semantics!)
```

Important: Projection removes duplicates in pure relational algebra, but SQL's SELECT does NOT (unless you use SELECT DISTINCT). SQL uses multiset semantics, not set semantics. This is a common source of confusion.

4.3 Set Operations

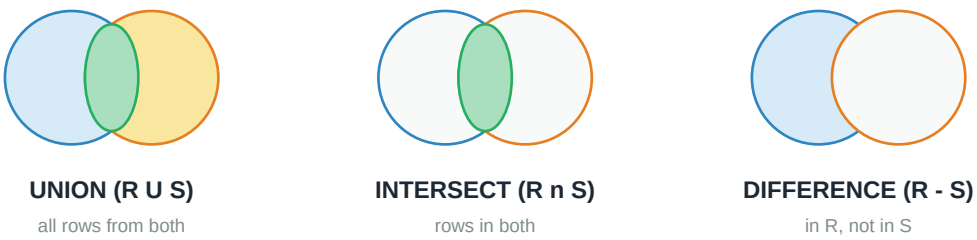


Figure: Union, Intersection, Difference — need union-compatible operands

Set operations require the two relations to be **union-compatible**: same number of attributes, and corresponding attributes have the same domain.

Union (U)

`R U S`: all tuples in R or S (or both). Duplicates removed.

Intersection (n)

`R n S`: tuples in both R and S. Derived from other operators: `R n S = R - (R - S)`.

Difference (-)

`R - S`: tuples in R but NOT in S. Not commutative — order matters.

4.4 Cartesian Product and Joins

The **Cartesian product** R X S pairs every tuple of R with every tuple of S. If R has m tuples and S has n, the result has m*n tuples with all attributes of both. Almost never useful on its own — but essential as the foundation of join operations.

Join Operations

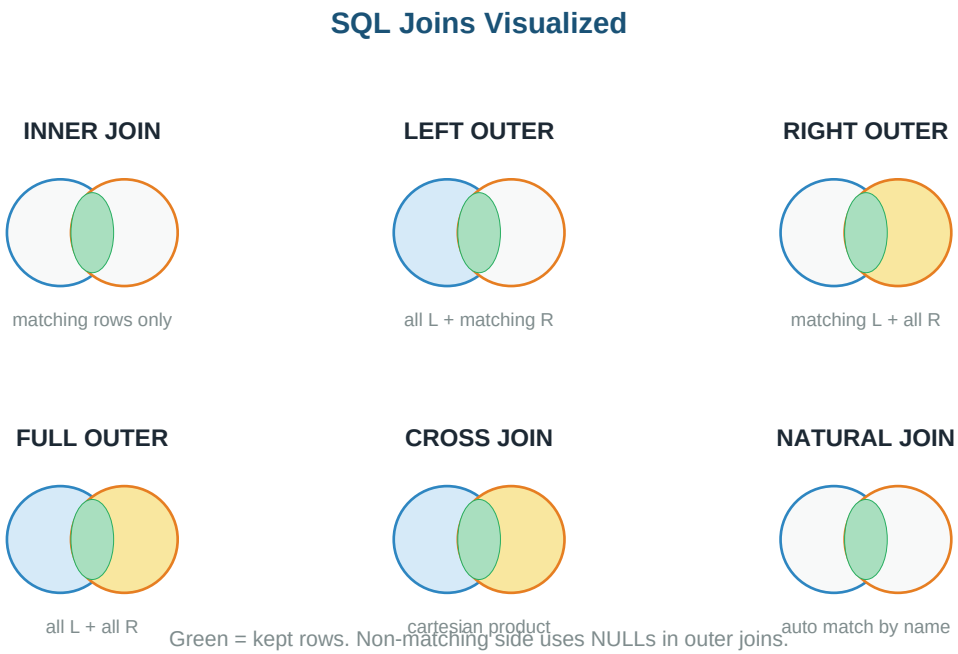


Figure: Different join types — colored regions show which rows are kept

Theta Join

A Cartesian product followed by a selection: `R X_{condition} S = sigma_condition(R X S)`. The condition can be any predicate involving attributes from both relations.

Equi Join

A theta join where the condition is an equality between attributes. E.g., `STUDENT X_{sid=roll} ENROLL`.

Natural Join (X_N)

An equi join on ALL attributes with the same name in both relations, followed by removing the duplicated columns. If STUDENT has sid and ENROLL has sid, natural join matches on sid and keeps only one sid column. Very common in practice.

Outer Joins

Regular joins drop rows that don't have a match. Outer joins keep them, filling missing attributes with NULLs:

- **Left Outer Join:** Keep all tuples from left relation; right side gets NULLs where no match.
- **Right Outer Join:** Mirror image — keep all right tuples.
- **Full Outer Join:** Keep all tuples from both sides.

Key Insight: Use LEFT OUTER when you want "all students, including those without enrollments". Use INNER JOIN when you want "students who actually enrolled". The choice is about what to do with unmatched rows.

4.5 Division

Division (R / S) is used for "for all" queries. It finds tuples in R that are related to ALL tuples in S.

Example: R(sid, cid) has student-course enrollments. S(cid) has all CS courses. R/S gives the sid of students who enrolled in EVERY CS course. Powerful, but SQL has no direct equivalent — you emulate it with double NOT EXISTS.

4.6 Relational Calculus

Where relational algebra says HOW to compute a query, **relational calculus** says WHAT you want. It's declarative — you describe the result, and it's the system's job to figure out how to get it. There are two flavors.

Tuple Relational Calculus (TRC)

Uses tuple variables. General form: `{ t | P(t) }` — the set of tuples t such that predicate P(t) is true. The predicate can quantify with FOR ALL and THERE EXISTS.

```
{ t | t IN STUDENT AND t.age > 20 }
-> all student tuples with age > 20

{ t | THERE EXISTS s IN STUDENT ( t.name = s.name AND s.age < 21 ) }
-> names of students under 21
```

Domain Relational Calculus (DRC)

Similar to TRC but uses domain variables — one per attribute — instead of tuple variables. QBE (Query-By-Example) is based on DRC. Both TRC and DRC are equally expressive as relational algebra.

Key Insight: SQL is essentially a hybrid — declarative like calculus, but with grouping, ordering, and other extensions from practical needs. Understanding algebra helps you predict what the SQL optimizer will do; understanding calculus helps you think about queries in terms of the result you want.

Chapter 5

SQL — Structured Query Language

SQL is the language of relational databases. It's declarative, standardized, and readable. This chapter walks through its building blocks — from creating tables to complex nested queries.

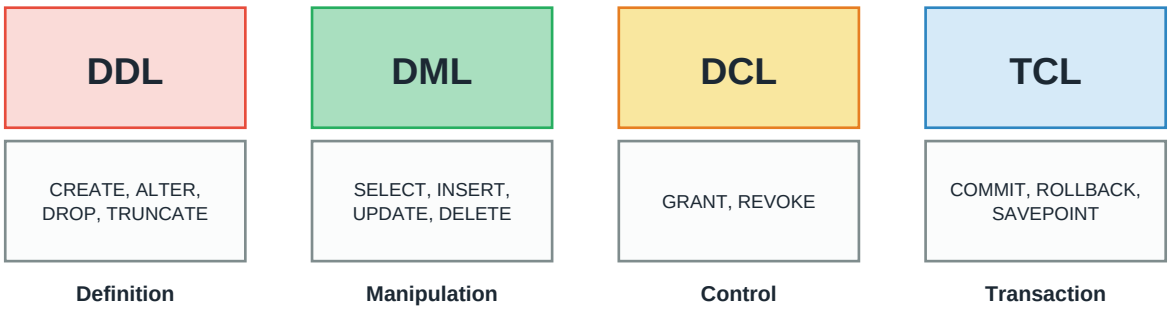
5.1 What Makes SQL Special

SQL (Structured Query Language) is the ANSI/ISO standard language for relational databases. Introduced by IBM in the 1970s (as SEQUEL), it became a formal standard in 1986 and has been the dominant database language ever since.

SQL is **declarative**: you say WHAT you want, not HOW to get it. "Give me all students in CS with GPA above 8" — you don't tell the database whether to scan the table, use an index, or do something clever. That's the job of the **query optimizer**, which converts your SQL into an efficient execution plan.

> ***Analogy:*** *Ordering food at a restaurant is declarative: "I'll have the pasta." You don't specify the cooking steps. Compare that to a recipe (procedural): "boil water, then add salt, then add pasta...". SQL is the menu; execution plans are the kitchen work. This separation is why the same SQL can run efficiently on different databases — each optimizer figures out its own best plan.*

SQL Sublanguages



SQL = Structured Query Language | Standard for RDBMSes since 1986

Some texts also list DQL (Data Query Language) for SELECT statements

Figure: Four categories of SQL commands

5.2 DDL — Defining the Schema

Data Definition Language creates and modifies the structure of the database.

CREATE TABLE

```
CREATE TABLE STUDENT (
sid INT PRIMARY KEY,
name VARCHAR(50) NOT NULL,
age INT CHECK (age >= 0),
dept VARCHAR(10) DEFAULT 'CS',
email VARCHAR(100) UNIQUE
);
```

Constraints go in the column definition. Common ones: **PRIMARY KEY, NOT NULL, UNIQUE, CHECK, DEFAULT, FOREIGN KEY**. Foreign keys can be inline (short form) or as a separate clause (with named constraints, useful for altering later).

ALTER TABLE

Modify an existing table without recreating it. Common operations:

- `ALTER TABLE STUDENT ADD COLUMN cgpa DECIMAL(3,2);`
- `ALTER TABLE STUDENT DROP COLUMN age;`
- `ALTER TABLE STUDENT MODIFY name VARCHAR(100);`
- `ALTER TABLE STUDENT ADD CONSTRAINT fk_dept FOREIGN KEY (dept) REFERENCES DEPARTMENT(did);`

DROP vs TRUNCATE vs DELETE

These sound similar but do very different things:

Command	What it does	Structure	Rollback?	Speed
DROP TABLE	Removes table entirely	Gone	No (DDL)	Fast
TRUNCATE	Removes all rows	Preserved	No (usually)	Very fast
DELETE	Removes matching rows	Preserved	Yes	Slow (per row)

Key Insight: TRUNCATE deallocates data pages in one go — that's why it's fast but can't be rolled back easily. DELETE processes row by row, respects triggers, and is fully transactional.

5.3 DML — The SELECT Statement

SELECT is where you'll spend 90% of your time. Understanding its full syntax and semantics is essential.

```
SELECT [DISTINCT] column_list
FROM table(s)
WHERE row_predicate
GROUP BY column_list
HAVING group_predicate
ORDER BY column_list [ASC|DESC]
LIMIT n;
```

Written Order vs Execution Order

You write SELECT first, but the database executes it LAST. Understanding execution order helps you reason about which clauses can reference what.

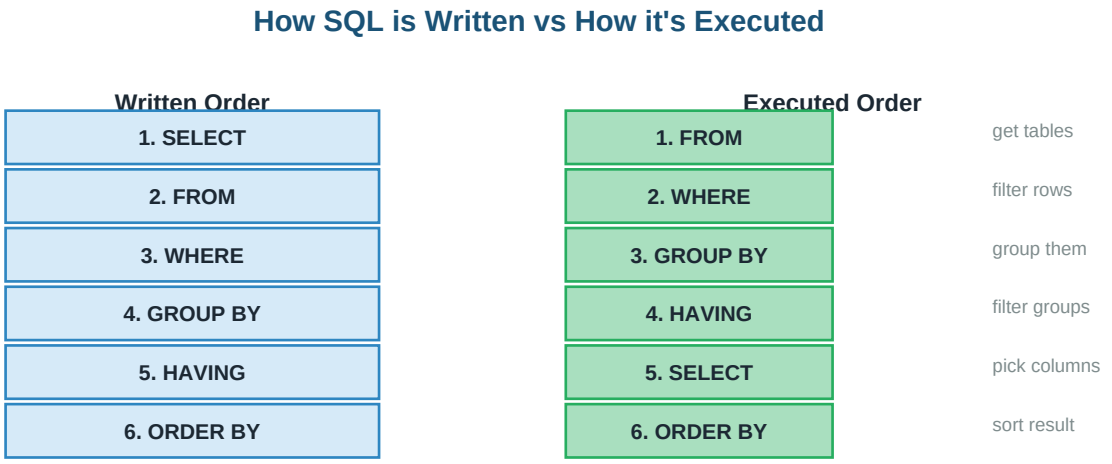


Figure: Why you can't use a SELECT alias in WHERE — SELECT hasn't executed yet

Important: This is why `SELECT age*2 AS double_age FROM students WHERE double_age > 40` usually fails: the WHERE runs before SELECT, so double_age doesn't exist yet. You must write `WHERE age*2 > 40` or use a subquery.

Filtering with WHERE

Predicates in WHERE support:

- **Comparisons:** =, <>, <, >, <=, >=
- **Logical:** AND, OR, NOT
- **Ranges:** `BETWEEN 10 AND 20`
- **Set membership:** `IN ('CS', 'EE', 'ME')`
- **Pattern matching:** `LIKE 'A%'` (% = any string, _ = one char)
- **NULL check:** `IS NULL`, `IS NOT NULL`

Important: NULL is not a value — it's the absence of a value. `NULL = NULL` is not TRUE; it's UNKNOWN. Always use `IS NULL`. Same for arithmetic: anything combined with NULL becomes NULL, which is a common source of bugs.

5.4 Joins

Joins connect data from multiple tables. Since we've seen the theory in Chapter 4, here we focus on SQL syntax.

Inner Join

```
SELECT s.name, c.title
FROM STUDENT s
JOIN ENROLL e ON s.sid = e.sid
JOIN COURSE c ON e.cid = c.cid;
```

Keeps only rows where the ON condition matches. Students who didn't enroll in any course don't appear. The old comma syntax (`FROM STUDENT s, ENROLL e WHERE s.sid = e.sid`) does the same thing but is discouraged — it's easy to forget the WHERE and get a huge Cartesian product.

Outer Joins

Include rows even when there's no match. Missing values become NULL.

```
LEFT OUTER JOIN -> all left rows, NULLs for unmatched right
RIGHT OUTER JOIN -> all right rows, NULLs for unmatched left
FULL OUTER JOIN -> all rows, NULLs on either side
```

Example: "Show all students AND their enrollments if any":
`SELECT s.name, e.cid FROM STUDENT s LEFT JOIN ENROLL e ON s.sid = e.sid;`
Students without enrollments still appear with NULL in the cid column.

Self Join

Joining a table with itself, using two aliases. Useful when a table has recursive relationships.

```
-- Find employees who earn more than their manager
SELECT e.name, e.salary, m.name, m.salary
FROM EMPLOYEE e
JOIN EMPLOYEE m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

Cross Join

Cartesian product with no join condition. Almost always a mistake unless you specifically want every combination. `SELECT * FROM A CROSS JOIN B`.

5.5 Aggregation and GROUP BY

Aggregate functions combine many rows into a single summary value. The main ones:

- **COUNT(*)** — number of rows (including NULLs)
- **COUNT(column)** — number of non-NULL values in that column
- **COUNT(DISTINCT column)** — number of unique non-NULL values
- **SUM, AVG, MIN, MAX** — self-explanatory; all ignore NULL

GROUP BY

Aggregates over the whole table give one number. Often you want one number PER group. **GROUP BY** partitions the table into groups and applies aggregation per group.

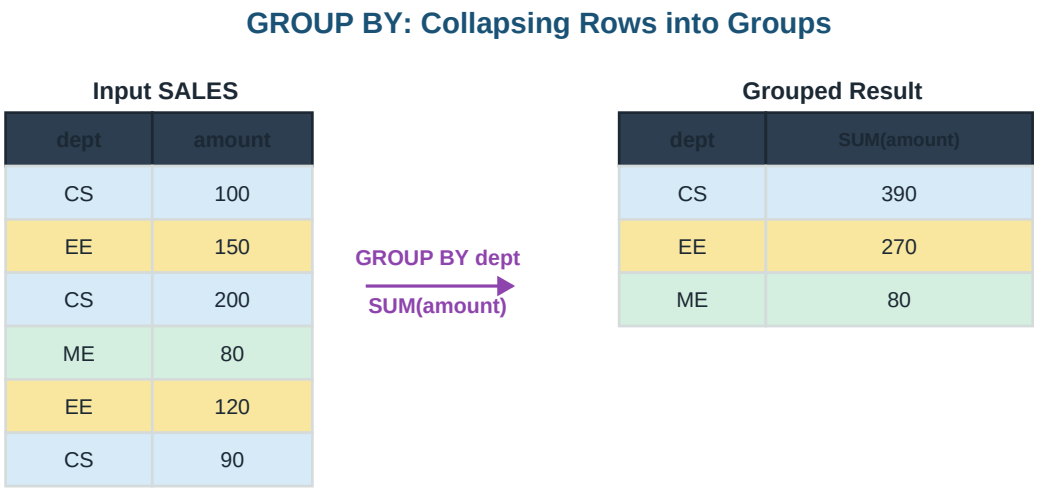


Figure: GROUP BY collapses rows sharing the group key

```
SELECT dept, COUNT(*), AVG(salary)
FROM EMPLOYEE
GROUP BY dept;
```

Important: Every non-aggregated column in SELECT must appear in GROUP BY. `SELECT dept, name, COUNT(*) FROM EMPLOYEE GROUP BY dept` is invalid — which name would the DB show for a group of employees?

HAVING vs WHERE

Both filter, but at different stages. **WHERE** filters INDIVIDUAL ROWS before grouping. **HAVING** filters GROUPS after aggregation. You can't put aggregates in WHERE because groups don't exist yet.

```
SELECT dept, AVG(salary) AS avg_sal
FROM EMPLOYEE
WHERE hire_year >= 2020 -- filter rows first
GROUP BY dept
HAVING AVG(salary) > 60000; -- filter groups next
```

5.6 Subqueries

A **subquery** is a SELECT inside another statement. Subqueries appear in WHERE, FROM, SELECT — almost anywhere an expression can go. They enable powerful multi-step logic in a single query.

Types of Subqueries

- **Scalar subquery:** Returns a single value. Used where a single value is expected. `SELECT name FROM STUDENT WHERE age = (SELECT MAX(age) FROM STUDENT);`
- **Row subquery:** Returns a single row with multiple columns.
- **Table subquery:** Returns a set of rows. Used with IN, EXISTS, ANY, ALL, or in FROM as a derived table.

Correlated vs Uncorrelated

An **uncorrelated** subquery can be executed independently — it doesn't reference the outer query. A **correlated** subquery references outer-query columns and must be re-evaluated for each outer row. Correlated subqueries are more expressive but often slower — the optimizer may or may not manage to rewrite them as joins.

```
-- Uncorrelated (subquery runs once):
SELECT name FROM EMPLOYEE
WHERE salary > (SELECT AVG(salary) FROM EMPLOYEE);

-- Correlated (runs once per outer row):
SELECT name FROM EMPLOYEE e
WHERE salary > (SELECT AVG(salary) FROM EMPLOYEE WHERE dept = e.dept);
```

EXISTS and NOT EXISTS

EXISTS is TRUE if the subquery returns at least one row. Common pattern for "does anything match this condition" queries.

```
-- Students who have enrolled in at least one course:
SELECT name FROM STUDENT s
WHERE EXISTS (SELECT 1 FROM ENROLL e WHERE e.sid = s.sid);

-- "For all" via double NOT EXISTS:
-- Students who took EVERY CS course:
SELECT name FROM STUDENT s
WHERE NOT EXISTS (
SELECT * FROM COURSE c WHERE c.dept = 'CS'
AND NOT EXISTS (SELECT * FROM ENROLL e WHERE e.sid = s.sid AND e.cid = c.cid)
);
```

5.7 Views

A **view** is a named query — it looks like a table but is really a stored SELECT. When you query the view, the DB substitutes the underlying SELECT.

```
CREATE VIEW cs_students AS
SELECT sid, name, cgpa FROM STUDENT WHERE dept = 'CS';

-- Now use it like a table:
SELECT * FROM cs_students WHERE cgpa > 8;
```

Why use views?

- **Security:** Grant users access to a view instead of the underlying tables.
- **Simplicity:** Hide complex joins behind a simple name.
- **Logical data independence:** Change the underlying tables without breaking apps that use the view.

Materialized views physically store the result and refresh periodically. Faster to query but may show stale data. Regular views recompute every time — always current but slower.

Chapter 6

Normalization

You have a database with duplicated data. It works — until an update leaves the copies inconsistent. Normalization is a systematic technique for restructuring tables to eliminate redundancy and its problems. This is easily the most tested topic in DBMS interviews and GATE.

6.1 The Problem: Anomalies from Redundancy

Suppose we store student enrollment in one big table: **sid, sname, cid, cname, instructor**. Every row has the student's info, the course's info, and the instructor. If a student takes 5 courses, their name appears 5 times. If 100 students take the same course, the instructor's name appears 100 times.

Anomalies from Redundancy

STUDENT_COURSE table (unnormalized)

sid	sname	course	instructor
101	Alice	OS	Dr. Ravi
101	Alice	DBMS	Dr. Sinha
102	Bob	OS	Dr. Ravi
103	Carol	OS	Dr. Ravi

Update: 'Dr. Ravi' quits - need to update ALL matching rows
Insert: cannot add a new course without at least one student
Delete: removing Bob's row loses 'OS taught by Dr. Ravi' info

Figure: Data anomalies caused by redundancy

This redundancy causes three types of anomalies:

- **Update Anomaly:** If Dr. Ravi quits and is replaced by Dr. Patel, we have to update every row where Dr. Ravi appears. Miss one, and now the table says both people teach the same course.
- **Insertion Anomaly:** Can't record a new course until at least one student enrolls in it — the table has no row for the course alone. This is nonsense: we should be able to define courses independently.
- **Deletion Anomaly:** If Bob was the only student enrolled in a course and Bob withdraws, deleting Bob's row also loses the fact that the course exists and who teaches it.

Key Insight: Normalization is really about breaking one big table into several smaller tables, each with a clear purpose. Then joins let us recombine information when needed — without redundancy.

6.2 Functional Dependencies

The formal engine driving normalization is the concept of a **functional dependency (FD)**. An FD captures the idea: "if I know the value of X, I automatically know the value of Y".

Written as `X -> Y`, read "X functionally determines Y". Formally: for any two tuples with the same X value, they must have the same Y value.

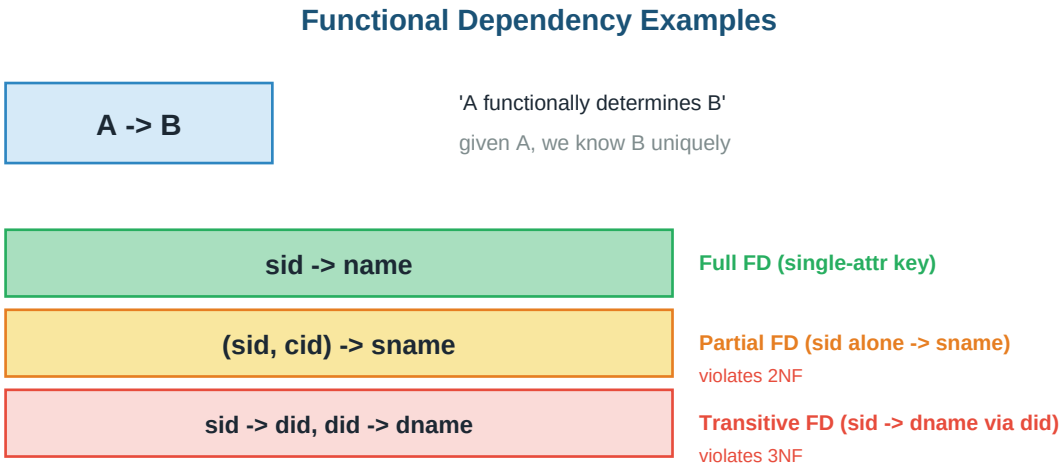


Figure: FDs — full, partial, and transitive

Example: In STUDENT(sid, name, dept, dept_head):
- `sid -> name`: same sid always has same name.
- `dept -> dept_head`: same dept always has same head.
- `sid -> dept_head`: derived — knowing sid gives dept gives head.

Types of Functional Dependencies

- **Trivial FD:** Y is a subset of X. E.g., `{sid, name} -> sid`. Always holds; not useful.
- **Non-trivial FD:** Y contains at least one attribute not in X. These are the interesting ones.
- **Full FD:** Removing any attribute from X breaks the dependency. `{sid, cid} -> grade` is full because neither sid alone nor cid alone determines grade.
- **Partial FD:** Some proper subset of X still determines Y. `{sid, cid} -> name` is partial because sid alone determines name.
- **Transitive FD:** X -> Y and Y -> Z gives X -> Z transitively. `sid -> dept, dept -> head` implies `sid -> head` transitively.

Armstrong's Axioms

The rules for deriving new FDs from a given set. Three fundamental ("sound and complete"):

- **Reflexivity:** If Y is a subset of X, then X -> Y.
- **Augmentation:** If X -> Y, then XZ -> YZ for any Z.
- **Transitivity:** If X -> Y and Y -> Z, then X -> Z.

Three additional (derived from the above):

- **Union:** If X -> Y and X -> Z, then X -> YZ.
- **Decomposition:** If X -> YZ, then X -> Y and X -> Z.
- **Pseudo-transitivity:** If X -> Y and WY -> Z, then WX -> Z.

6.3 Attribute Closure

The **closure** of a set of attributes X, denoted X^+, is the set of all attributes that X can determine using the given FDs. Closure is the most useful tool in normalization problems — you use it to find candidate keys, check FD membership, and verify normal forms.

Algorithm to Compute X^+

- **1.** Start with $X^+ = X$.
- **2.** Look at each FD Y -> Z in your set. If Y is a subset of X^+, add Z to X^+.
- **3.** Repeat until X^+ stops growing.

Example: Given R(A, B, C, D, E) with FDs {A -> B, B -> C, CD -> E}. Find A^+:
- Start: {A}
- A -> B gives {A, B}
- B -> C gives {A, B, C}
- CD -> E: need D too, don't have it. Stop.
- Answer: A^+ = {A, B, C}

Finding Candidate Keys

An attribute set X is a candidate key iff X^+ = all attributes AND no proper subset of X has the same closure. So the algorithm is: compute closure, check if it covers everything, then verify minimality.

Key Insight: Shortcut: attributes that appear only on the LEFT of FDs (never on the right) must be in every candidate key. Attributes that appear only on the RIGHT can never be in a candidate key alone.

6.4 Normal Forms

The normal forms are progressive rules for structuring relations. Each form is a stricter subset of the previous. Meeting a higher normal form implies meeting all lower ones.

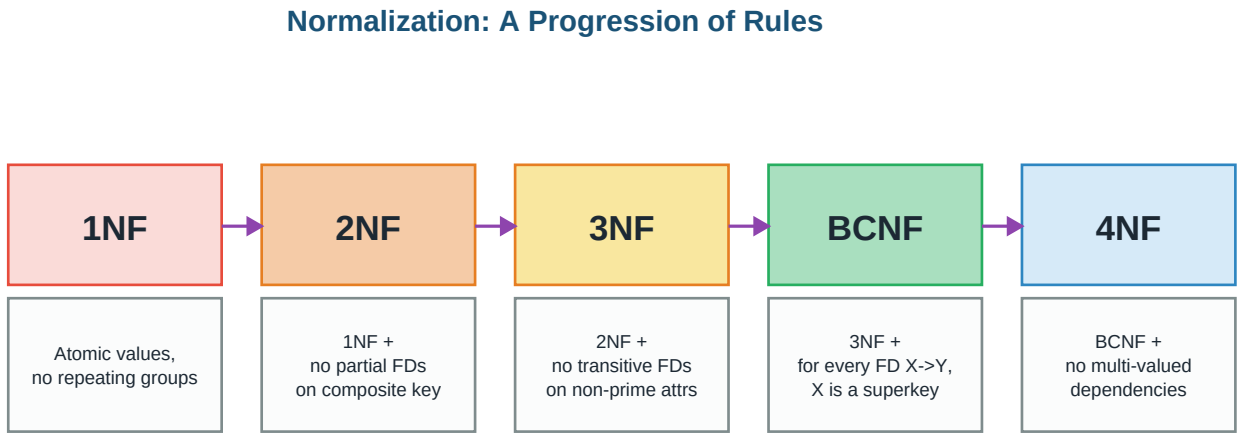


Figure: Progression of normal forms — each is stricter than the last

First Normal Form (1NF)

A relation is in 1NF if every attribute is **atomic** — no multivalued attributes, no composite attributes, no nested tables. Every cell holds a single value.

> **Example:** STUDENT(sid, name, phones) where phones = {"9999", "8888"} violates 1NF. Fix by splitting into two relations: STUDENT(sid, name) and STUDENT_PHONE(sid, phone).

1NF is the entry ticket to the relational model. In practice, if you're storing lists in a comma-separated string, you've broken 1NF.

Second Normal Form (2NF)

A relation is in 2NF if it's in 1NF AND has no **partial dependencies** — no non-prime attribute depends on a proper subset of any candidate key. (A **non-prime attribute** is one that's not part of any candidate key. A **prime attribute** is part of at least one candidate key.)

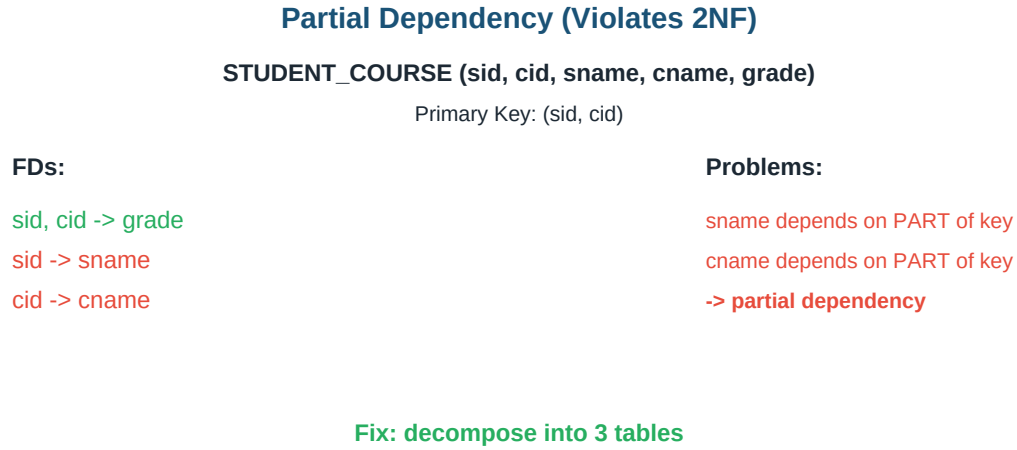


Figure: Partial dependency violates 2NF

2NF only matters when the primary key is **composite**. If your primary key is a single attribute, 2NF is automatically satisfied — a single attribute has no proper subset (other than the empty set).

Third Normal Form (3NF)

A relation is in 3NF if it's in 2NF AND has no **transitive dependency** from a candidate key to a non-prime attribute. Formally: for every non-trivial FD X -> A, either X is a superkey, or A is a prime attribute.

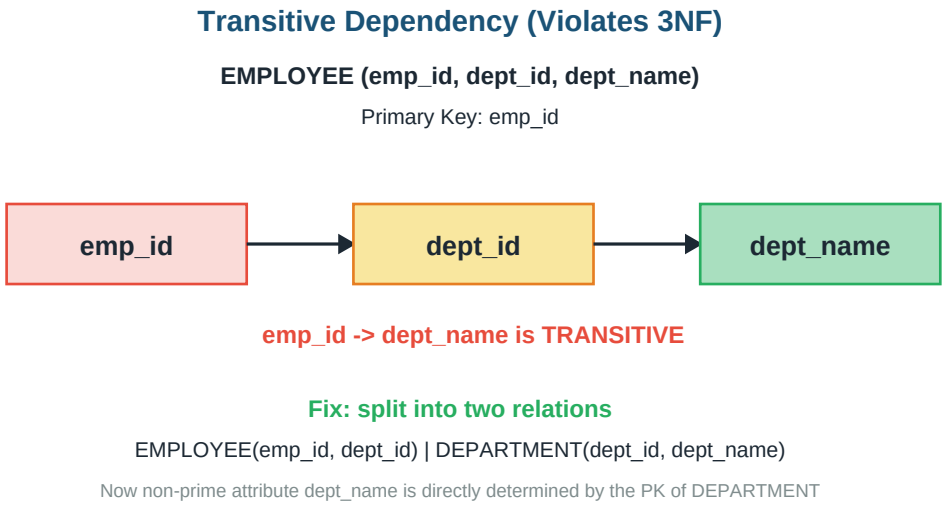


Figure: Transitive dependency violates 3NF

Example: EMPLOYEE(emp_id, dept_id, dept_name) with FDs emp_id -> dept_id and dept_id -> dept_name. emp_id transitively determines dept_name via dept_id — 3NF violated. Fix: split into EMPLOYEE(emp_id, dept_id) and DEPARTMENT(dept_id, dept_name).

Boyce-Codd Normal Form (BCNF)

The strictest form for most practical purposes. A relation is in BCNF if **for every non-trivial FD X -> A, X is a superkey**. This is stricter than 3NF because it doesn't allow the "A is prime" escape hatch.

Example: R(A, B, C) with FDs {AB -> C, C -> B}. Candidate keys: {A, B} and {A, C}. So A, B, C are all prime attributes. The FD C -> B has C on the left — but C is not a superkey. Violates BCNF. But it's in 3NF because B is a prime attribute (member of candidate key AB).

Key Insight: Every BCNF is 3NF, but not every 3NF is BCNF. In practice most designs aim for BCNF. But BCNF decomposition might lose dependency preservation — see next section.

Fourth Normal Form (4NF)

Deals with **multi-valued dependencies (MVDs)**: independent multi-valued facts about the same entity. A relation is in 4NF if it's in BCNF and has no non-trivial MVDs.

Example: STUDENT_HOBBY_LANGUAGE(sid, hobby, language) where a student has multiple hobbies AND multiple languages, and hobbies and languages are independent. This produces a Cartesian-product-like table. Fix: split into STUDENT_HOBBY(sid, hobby) and STUDENT_LANGUAGE(sid, language).

6.5 Properties of Decomposition

When we break a relation R into R1 and R2 (or more), we need to check two properties.

Lossless-Join Decomposition

The decomposition is **lossless** if joining R1 and R2 back gives exactly R — no fewer, no more tuples. If joining produces spurious tuples (rows that were never in the original), the decomposition is **lossy** and destroys information.

Lossless-Join Decomposition

Original R (A, B, C)

A	B	C
a1	b1	c1
a2	b2	c1

R1 (A, B) and R2 (B, C)

A	B
a1	b1
a2	b2

B	C
b1	c1
b2	c1

Lossless iff R1 n R2 -> R1 OR R1 n R2 -> R2

(common attributes must be a superkey of at least one part)

If lossy: joining R1 and R2 gives spurious tuples not in R

Figure: Test for losslessness — intersection must be a superkey of at least one relation

The formal test: decomposition of R into R1 and R2 is lossless if and only if **(R1 n R2) -> R1** or **(R1 n R2) -> R2** — the common attributes must be a superkey of at least one part.

Dependency-Preserving Decomposition

The decomposition is **dependency-preserving** if all the original FDs can be enforced by checking each new relation independently — without expensive joins to check across relations.

Example: Original R(A, B, C) with FDs A -> B and B -> C. Decompose into R1(A, B) and R2(A, C). A -> B stays within R1 (good). But B -> C is lost — B is in R1, C is in R2, so to enforce B -> C we'd have to join. Not dependency-preserving.
Better: R1(A, B) and R2(B, C). Both FDs preserved.

Important: Ideal decomposition is **both lossless and dependency-preserving**. Any relation can be decomposed to 3NF while preserving both. But BCNF decomposition is always lossless — but sometimes NOT dependency-preserving. This is the fundamental trade-off between 3NF and BCNF.

Chapter 7

Transactions and Concurrency Control

Real databases serve many users at once. How do we ensure their operations don't corrupt each other's data — while still running things fast? This chapter covers transactions, the ACID properties, and the mechanisms that make concurrent database access safe.

7.1 What is a Transaction?

A **transaction** is a sequence of database operations that logically belongs together and must be treated as a single unit. Classic example: transferring money from account A to account B involves TWO updates — debit A, credit B. If a crash happens between them, one account changed and the other didn't. That's disaster.

A transaction gives us a way to say "these operations happen together or not at all". The database guarantees this with the four **ACID properties**.

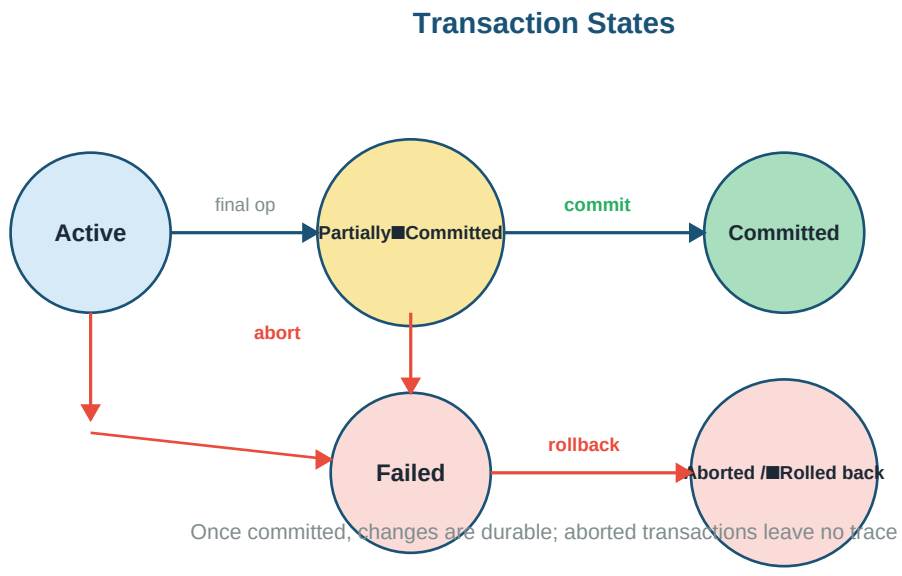


Figure: A transaction goes through several states from active to committed or aborted

Transaction States

- **Active:** Currently executing operations.
- **Partially Committed:** Last statement finished; not yet made permanent.
- **Committed:** Changes durably saved. Cannot be undone.
- **Failed:** Something went wrong — constraint violation, disk full, etc.
- **Aborted / Rolled Back:** All changes undone; database returned to state before txn started.

7.2 ACID Properties

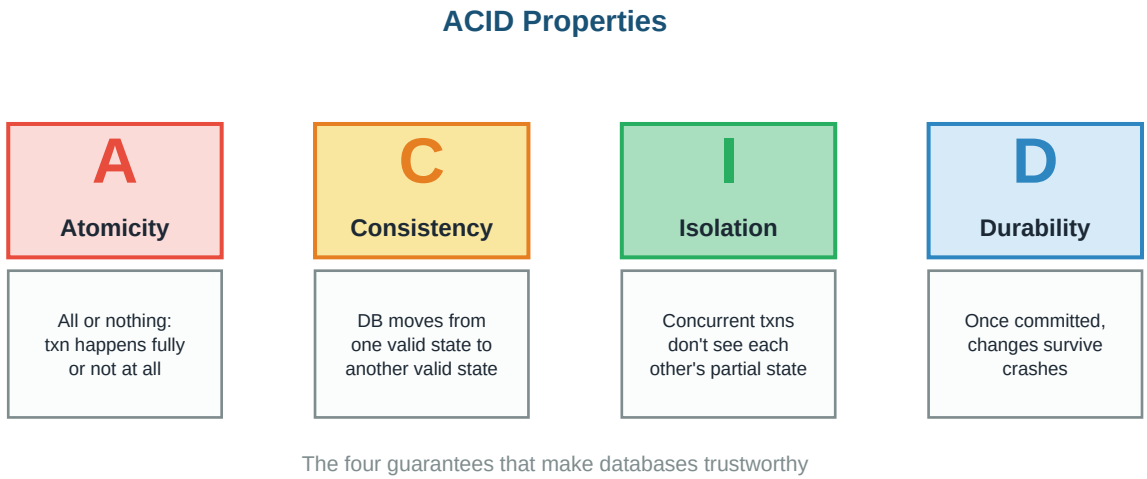


Figure: ACID — the four guarantees that make databases trustworthy

Atomicity

All or nothing. Either every operation in the transaction happens, or none of them do. There's no in-between visible to anyone else. If any part fails, the whole transaction is rolled back.

> ***Analogy:*** *Booking a flight: reserve the seat, charge the card, send confirmation email. If the card charge fails, we shouldn't reserve the seat. Atomicity handles this — the transaction manager undoes any partial work.*

Consistency

The database moves from one valid state to another. If integrity constraints hold before the transaction, they hold after. Referential integrity, check constraints, key constraints — all preserved.

Consistency is partly enforced by the DBMS (constraint checks) and partly the application's job (the code inside the transaction must make sense).

Isolation

Concurrent transactions don't see each other's partial changes. To each transaction, the database looks like it's the only one running — even though many transactions may execute simultaneously in reality.

True full isolation is expensive (essentially serial execution). Real databases offer weaker **isolation levels** that trade some isolation for performance: READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE.

Durability

Once committed, changes survive. Even if the DB crashes immediately after commit, when it restarts, the committed data is still there. This is achieved through write-ahead logging (Chapter 8).

7.3 Schedules and Serializability

When multiple transactions run at the same time, the database interleaves their operations. The resulting interleaved sequence is called a **schedule**.

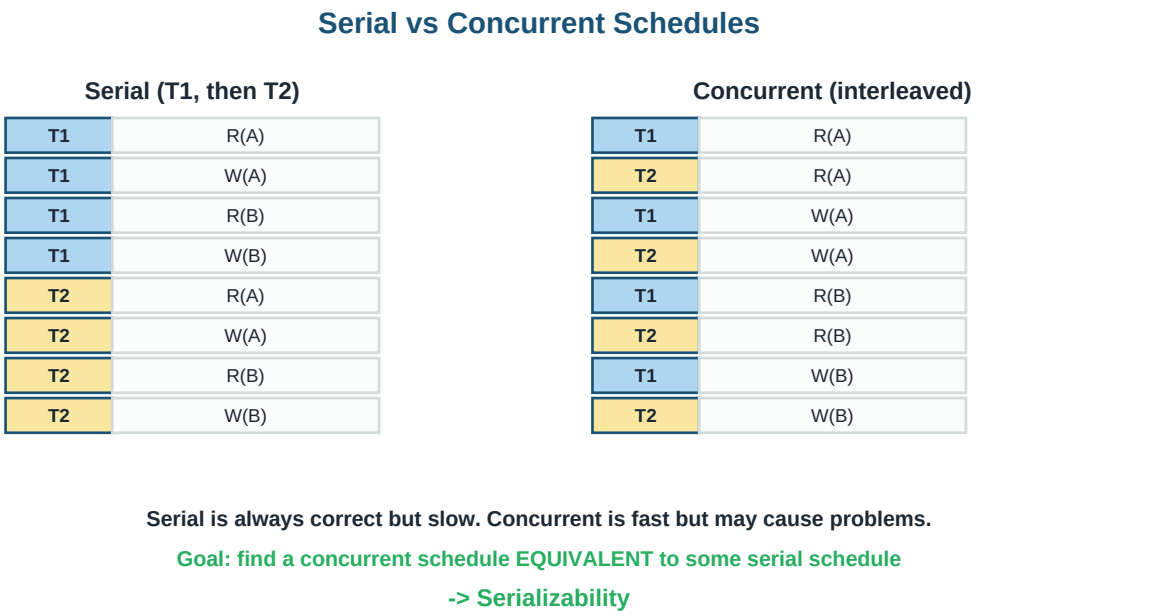


Figure: Serial schedule vs concurrent interleaving — need equivalence to reason about correctness

Serial vs Concurrent Schedules

A **serial schedule** runs transactions one at a time, no interleaving. Always safe but slow — the whole point of concurrency is to speed things up.

A **concurrent schedule** interleaves operations. Faster, but may produce results different from any serial order. We want concurrent schedules whose EFFECT is the same as SOME serial order — these are called **serializable**.

Conflict Serializability

Two operations conflict if:

- They belong to DIFFERENT transactions,
- They access the SAME data item,
- At least one of them is a WRITE.

A schedule is **conflict-serializable** if we can transform it into a serial schedule by swapping adjacent non-conflicting operations. Equivalently: draw a **precedence graph** — one node per transaction, edge Ti -> Tj whenever a Ti operation precedes a conflicting Tj operation. The schedule is conflict-serializable iff the graph is ACYCLIC.

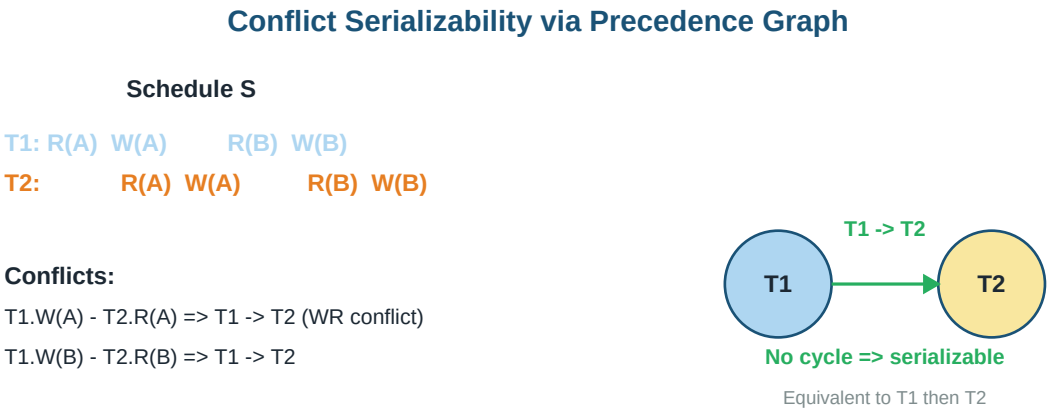


Figure: Precedence graph — cycle means not serializable

View Serializability

A weaker requirement. Two schedules are **view-equivalent** if:

- For each data item, the same transaction performs the initial read in both schedules.
- For each read, the transaction reads the value written by the same predecessor in both.
- The final write on each data item is done by the same transaction in both.

A schedule is view-serializable if it's view-equivalent to some serial schedule. Every conflict-serializable schedule is view-serializable, but not vice versa. Testing view serializability is NP-complete, so practical systems use conflict serializability.

7.4 Concurrency Control Protocols

How do we ensure schedules are serializable? Through **concurrency control** protocols — rules that transactions must follow. Two main families: lock-based and timestamp-based.

Lock-Based Protocols

Before accessing a data item, a transaction acquires a **lock** on it. Two lock types:

- **Shared (S) Lock:** For reading. Multiple transactions can hold S locks on the same item simultaneously — reads don't conflict with each other.
- **Exclusive (X) Lock:** For writing. No other transaction can hold ANY lock on the item while an X lock is held. Writes conflict with everything.

	S request	X request
S held	GRANT	WAIT
X held	WAIT	WAIT

Two-Phase Locking (2PL)

2PL is the most common protocol. Every transaction goes through two phases:

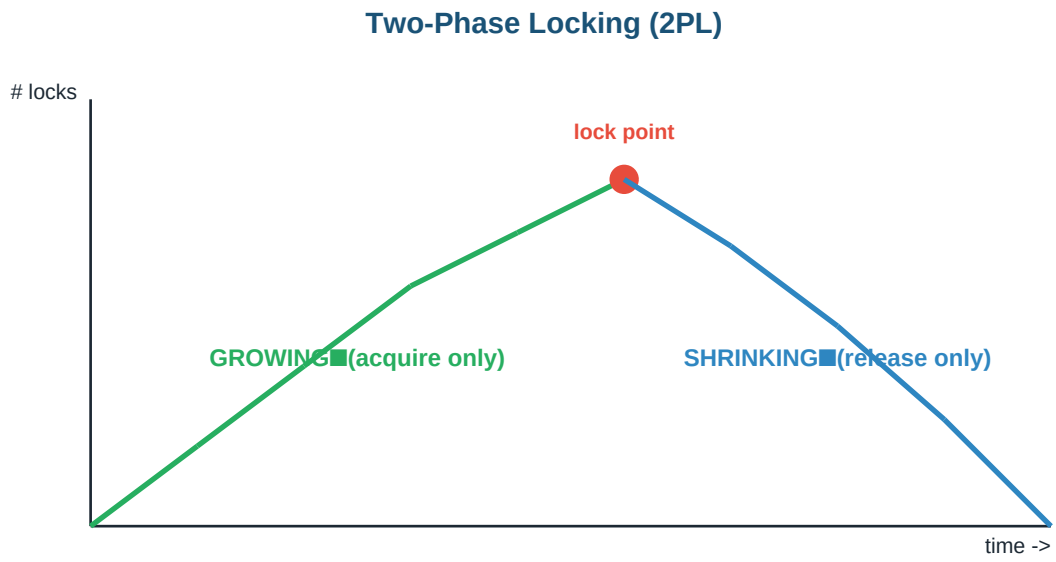


Figure: Two-phase locking — acquire everything, then release everything

- **Growing phase:** Transaction may acquire locks but cannot release any.
- **Shrinking phase:** Transaction may release locks but cannot acquire any new ones.

The transition point (after the last acquire, before the first release) is the **lock point**. 2PL guarantees conflict serializability — but it can cause deadlocks.

Strict and Rigorous 2PL

- **Strict 2PL:** All exclusive locks held until commit/abort. Prevents cascading rollbacks (if T1 aborts, its writes need to be undone; if T2 read those writes, T2 must also abort — cascading).
- **Rigorous 2PL:** ALL locks (both shared and exclusive) held until commit/abort. Most restrictive form. Used in practice by most commercial DBMSes.

Timestamp-Based Protocol

An alternative to locking. Each transaction gets a unique **timestamp** when it starts. Operations are ordered by timestamp — the serial equivalent order is timestamp order. Each data item stores the timestamp of the latest reader and writer.

When Ti wants to read/write item X, the protocol compares Ti's timestamp with the read/write timestamps of X. If Ti is "too late" (a younger transaction already read/wrote), Ti is aborted and restarted with a new timestamp.

> **Key Insight:** The **Thomas Write Rule** is an optimization: if Ti wants to write X but a younger transaction already wrote X, Ti's write is IGNORED (not aborted) — because whatever it wrote would be immediately overwritten anyway. Saves aborts.

7.5 Deadlocks in DBMS

The same deadlock problems from operating systems apply to databases. Two transactions each holding a lock the other needs — neither can proceed. In DB context, we usually reason with the **wait-for graph**.

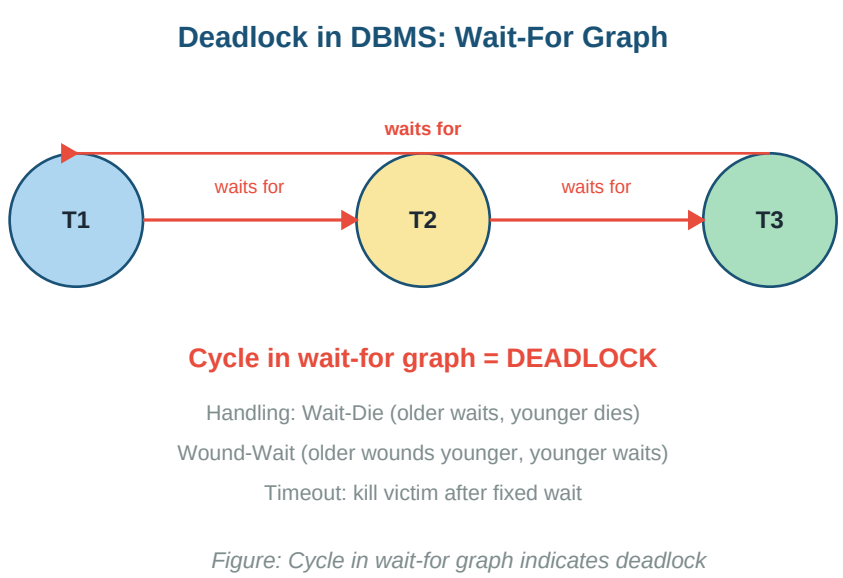


Figure: Cycle in wait-for graph indicates deadlock

Deadlock Prevention

Prevent deadlocks from forming in the first place, using transaction timestamps:

- **Wait-Die (non-preemptive):** If Ti requests a lock held by Tj: if Ti is OLDER, it waits; if younger, it dies (aborts and restarts with same timestamp). Older transactions have priority.
- **Wound-Wait (preemptive):** If Ti requests a lock held by Tj: if Ti is OLDER, it wounds Tj (Tj aborts); if younger, it waits. Again, older wins.

Key Insight: Both schemes prevent deadlock because they impose a partial order — older transactions always have priority. But they cause unnecessary restarts, so they're only worthwhile in high-contention systems.

Deadlock Detection and Recovery

Let deadlocks happen; detect them periodically by looking for cycles in the wait-for graph. When detected, choose a **victim** transaction and abort it. Victim selection considers: transaction age, amount of work done, number of items locked, and how many other transactions are affected.

Chapter 8

Recovery, File Organization & Indexing

The last chapter covers the storage side of DBMS: how data survives crashes, how it's laid out on disk, and how indexes make queries fast. Understanding these is critical for real performance work.

8.1 Recovery — Surviving Crashes

The database sits in RAM and disk. RAM is fast but volatile — a crash wipes it. Disk is slow but persistent. When a transaction commits, we need its changes to survive on disk. But we can't wait for every write to hit disk before continuing — that would be too slow.

The solution is **write-ahead logging (WAL)** — the foundation of nearly all recovery techniques.

Write-Ahead Logging

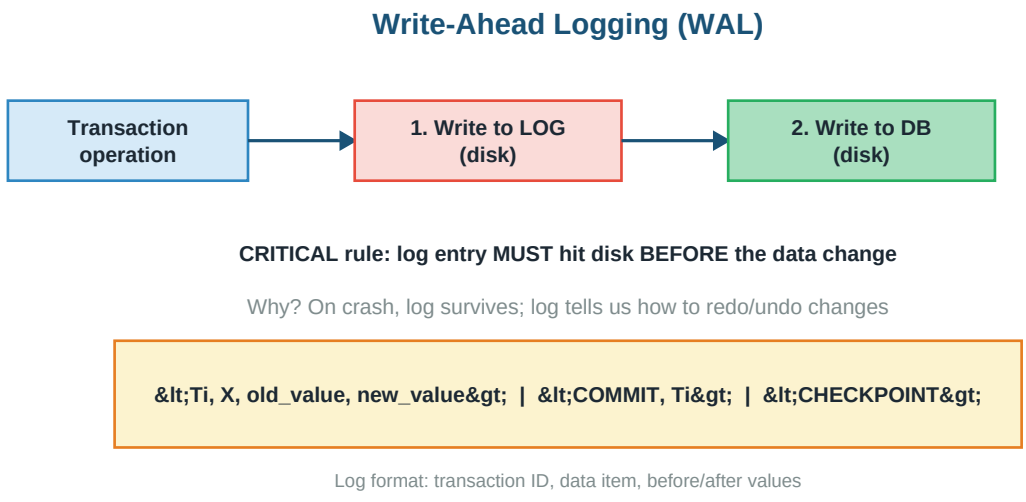


Figure: WAL — log entry hits disk before the data change

Every change is described in a log entry BEFORE the change itself is written to the actual database files. The rule: **the log entry must reach stable storage before the modified data page does**. This way, on a crash, we can reconstruct exactly what happened.

A log entry typically looks like:

```
<Ti, X, V_old, V_new> -- Ti updated X from V_old to V_new
<Ti, START> -- Ti started
<Ti, COMMIT> -- Ti committed
<Ti, ABORT> -- Ti aborted
<CHECKPOINT L> -- L = list of active transactions
```

Deferred vs Immediate Updates

- **Deferred update:** All changes buffered in memory until commit. On commit, log is written first, then changes applied to disk. No undo needed — nothing was written before commit. If crash: REDO committed transactions from log.
- **Immediate update:** Changes written to disk as they happen (in the background). Log entry logged first per WAL rule. If crash: UNDO uncommitted transactions AND REDO committed ones.

Checkpoints

The log can grow enormous. Replaying it entirely after a crash would take forever. **Checkpoints** solve this: periodically, the database flushes all dirty pages to disk and writes a checkpoint record to the log. After a crash, recovery only needs to replay from the last checkpoint.

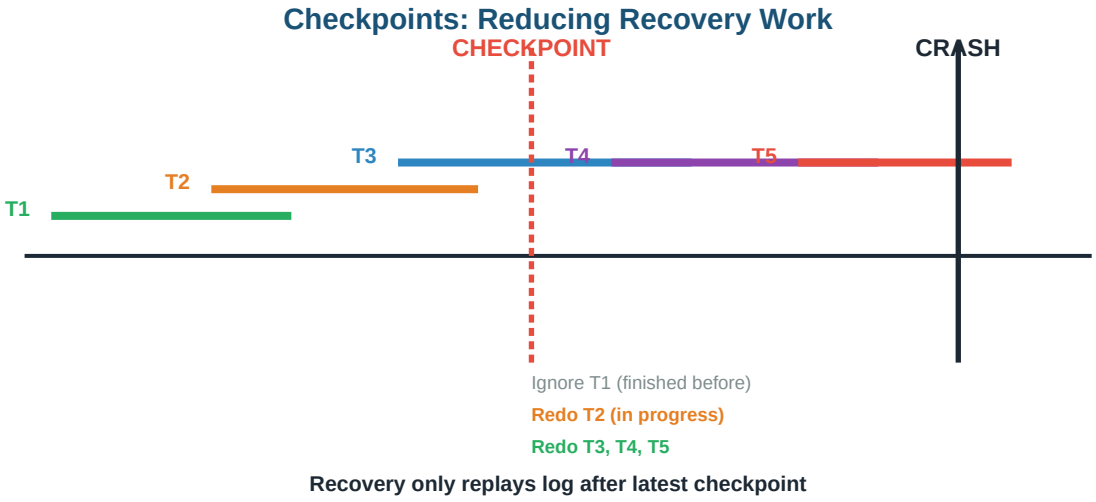


Figure: Checkpoint bounds the recovery work

Shadow Paging

A completely different approach that doesn't use logs. Maintain two page tables: the **current** and the **shadow**. Modifications go to new pages, updating only the current page table. On commit, atomically swap current and shadow (just one pointer change). On abort, discard current, keep shadow.

Simple and elegant — no undo/redo logic — but has drawbacks: doesn't work well with concurrent transactions, causes data fragmentation, and expensive garbage collection. Rarely used in practice.

8.2 File Organization

Data is ultimately stored in files on disk. How records are arranged inside files affects everything from insert speed to search performance.

Heap (Unordered) Files

Records placed anywhere in the file — usually at the end. Fastest inserts. But finding a specific record requires scanning the whole file: O(N). Deletes leave gaps. Good for tables that are appended to more than searched (log tables, staging tables).

Sequential (Ordered) Files

Records physically sorted by some key. Binary search: O(log N). Range scans are efficient. But inserts and updates that change the sort key are painful — you have to shift records or maintain overflow areas.

Hash Files

Records placed in buckets based on hash of a key. Direct access: O(1) average. Terrible for range queries — related values hash to different buckets.

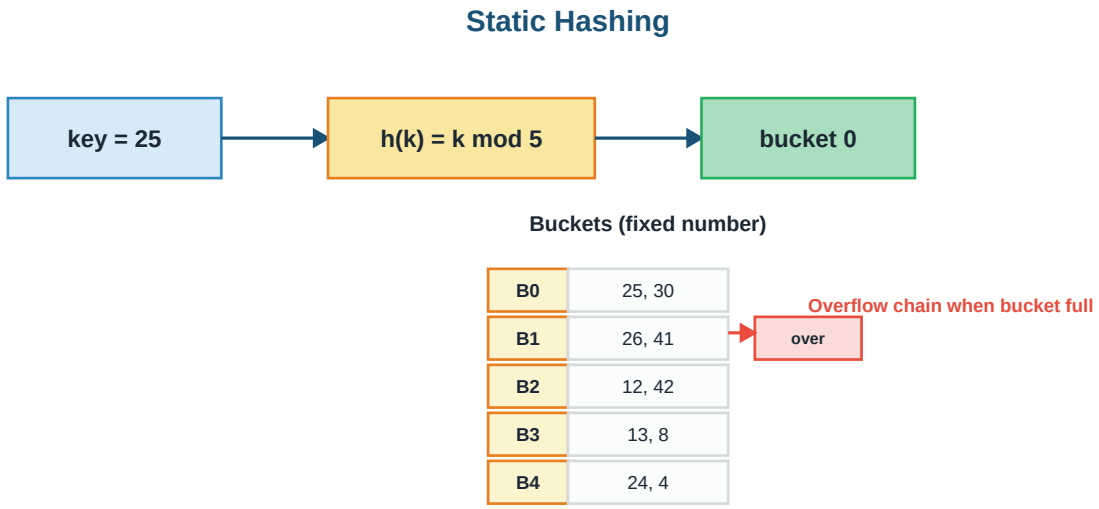


Figure: Static hashing — buckets fixed, overflow handling for collisions

Static vs Dynamic Hashing

Static hashing has a fixed number of buckets. As data grows, buckets overflow and performance degrades. **Dynamic hashing** (extendible hashing, linear hashing) grows the number of buckets smoothly as needed — using a directory of pointers to buckets, split when a bucket overflows.

8.3 Indexing

A file scan through a million-row table takes seconds. An **index** lets us find any specific row in microseconds by maintaining a separate, small data structure that maps search keys to record locations.

> ***Analogy:*** *A book's index at the back is exactly this: instead of reading every page to find mentions of "deadlock", you look up "deadlock" in the index and jump directly to the pages. Database indexes work the same way — but they can be built on any column, and updated as the data changes.*

Primary vs Secondary Index

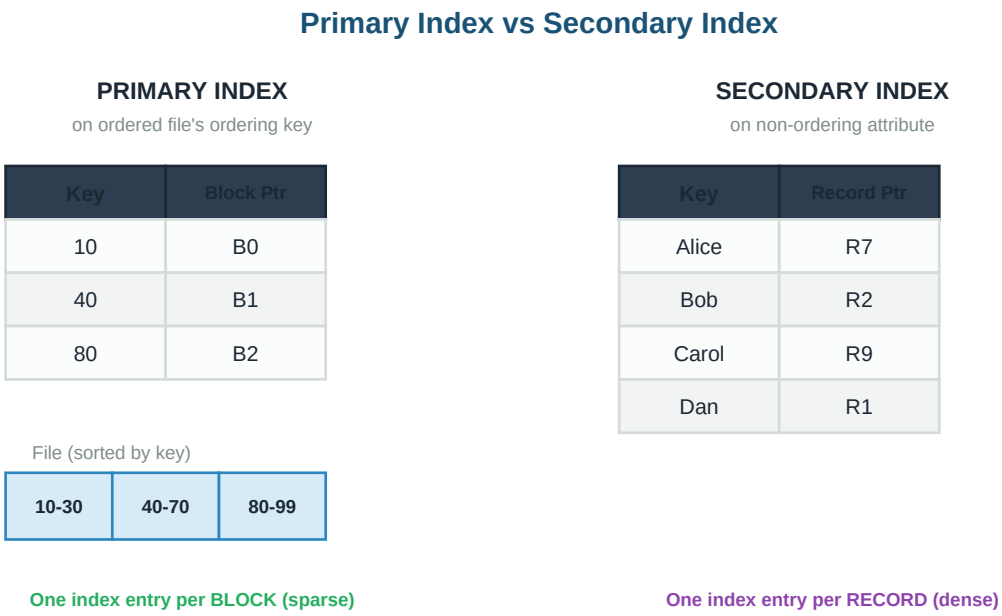
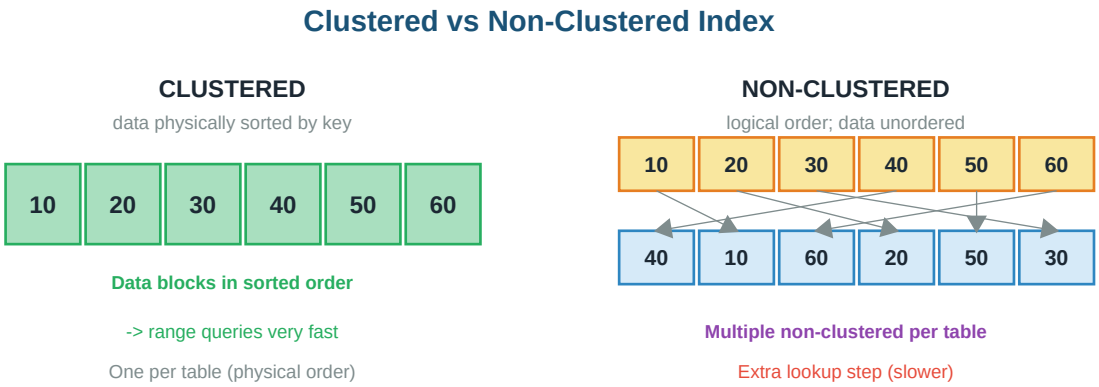


Figure: Primary index (sparse, on ordering key) vs secondary index (dense, on any attribute)

- **Primary index:** Built on the attribute that the file is physically sorted by. There's one entry per BLOCK — the index is sparse, so it's small. Only one primary index per table.
- **Secondary index:** Built on any other attribute. Has one entry per RECORD (dense), because records aren't sorted by this attribute. Larger than primary indexes. You can have many per table.
- **Clustering index:** Same as primary — data physically ordered by index key. Great for range queries.

Clustered vs Non-Clustered



Clustered = phone book (data sorted by name).

Non-clustered = book index (page numbers point into unsorted data).

Figure: Clustered = data sorted; Non-clustered = pointers to unsorted data

A **clustered index** determines the physical order of data. Only one per table. Range queries and lookups by the index key are very fast because related rows are adjacent on disk.

A **non-clustered index** is separate from the data — it stores index entries pointing to rows. You can have many. Each lookup requires reading the index, then following a pointer to the actual row — one extra I/O.

8.4 B-Trees and B+ Trees

Simple indexes work for small data but don't scale. Modern databases use tree-based indexes with high fan-out that stay balanced automatically. The B-tree family is the workhorse.

B-Tree

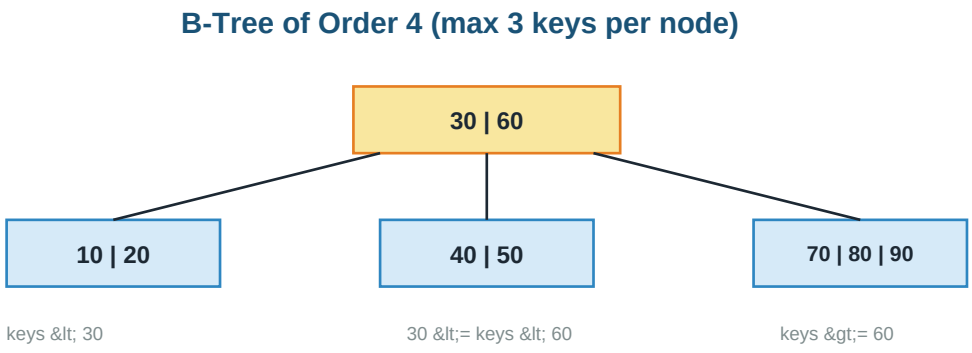


Figure: B-tree — balanced multi-way search tree

A B-tree of order m allows each node to have up to m children. Keys within a node are sorted. Between each pair of adjacent keys is a pointer to a child subtree containing keys in that range. All leaves at the same level — the tree is always balanced.

Search: O(log_m N). With m around 100+ (typical for disk pages), even a billion-row table has depth 5 or less — meaning 5 disk reads for any lookup. Insertions and deletions preserve balance by splitting or merging nodes as needed.

B+ Tree

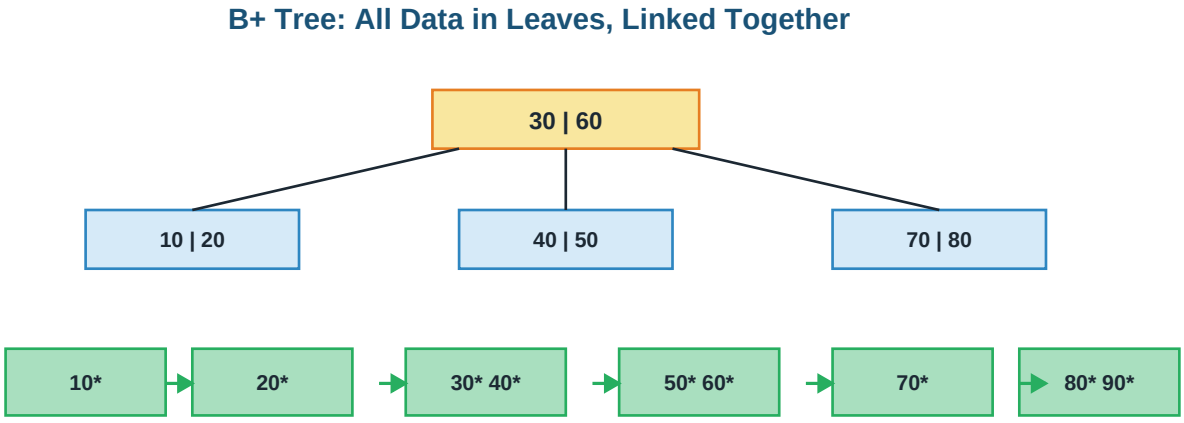


Figure: B+ tree — data only in leaves, leaves linked in a list

A B+ tree is a refinement of B-tree with two key differences:

- All actual data pointers are stored in the **leaves**. Internal nodes hold only keys for routing.
- Leaves are linked in a doubly-linked list. Range queries walk the linked list — no need to go back up and down the tree.

This design has advantages that make B+ trees dominant in real databases (MySQL InnoDB, PostgreSQL, Oracle, SQL Server, most others):

- **Higher fan-out** in internal nodes (no data means keys pack tighter) → shallower tree.
- **Fast range scans** via linked leaves.
- **Uniform query cost** — every lookup goes to the same depth.
- **Easier concurrency** — leaves change frequently but internal nodes stay stable.

Key Insight: When people say "database index", they almost always mean B+ tree. Even hash indexes are used less often because they don't support range queries. Understanding B+ trees is the single most valuable storage-side skill.

8.5 Other Index Types

Bitmap Index

For low-cardinality attributes (few distinct values), store one bit per row per possible value. E.g., for gender (M/F): two bitmaps, one for M rows, one for F. Very compact, extremely fast for AND/OR queries across multiple attributes — great for data warehouses. Terrible for high-cardinality attributes and for tables that update often.

Inverted Index

Standard in full-text search. For each term (word), store the list of documents (rows) that contain it. Google's index is essentially a massive inverted index. Enables queries like "documents containing 'database' AND 'index' but NOT 'sql'" to be answered by intersecting/union-ing lists.

Hash Index

Fast O(1) equality lookup, but doesn't support range queries or sorted access. Used for specific cases where only equality matters (memcached, some in-memory databases). Most disk databases prefer B+ trees for their flexibility.

Appendix

Quick Reference Formulas & Key Points

One-page revision reference. All the essential rules, notations, and comparisons in one place — perfect for a last look before exam or interview.

A.1 ER Notation Cheat Sheet

- **Rectangle** = entity set | **Double rectangle** = weak entity set
- **Diamond** = relationship | **Double diamond** = identifying relationship
- **Oval** = attribute | **Double oval** = multivalued | **Dashed oval** = derived
- **Underlined attribute** = key | **Dashed underline** = partial key (weak entity)
- **Single line** = partial participation | **Double line** = total participation

A.2 ER to Relational Rules

- **Strong entity**: new relation, entity's key as PK.
- **Weak entity**: new relation, PK = owner's PK + partial key.
- **1:1**: FK on either side (prefer total-participation side).
- **1:N**: FK on the N-side (referring to 1-side).
- **M:N**: new relation with FKs of both entities as composite PK.
- **Multivalued attribute**: separate relation with entity PK + attribute.

A.3 Keys

```
Super key ⊃= Candidate key ⊃= Primary key
Candidate key = minimal super key (no removable attribute)
Primary key = chosen candidate key (unique + NOT NULL)
Alternate key = candidate keys not chosen as PK
Foreign key = attribute referencing PK of another relation
```

A.4 Relational Algebra

```
sigma_{condition}(R) = select rows satisfying condition
pi_{attributes}(R) = project chosen columns (removes dups)
R U S = union (union-compatible required)
R n S = intersection = R - (R - S)
R - S = tuples in R not in S
R x S = cartesian product
R JOIN_c S = theta join = sigma_c(R x S)
R NATURAL JOIN S = equi-join on all same-named attrs
```

```
R / S = division ("for all")
```

A.5 SQL Execution Order

```
Written : SELECT -> FROM -> WHERE -> GROUP BY -> HAVING -> ORDER BY
Executed: FROM -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY
```

Key SQL rules:

- Cannot use SELECT aliases in WHERE (WHERE runs before SELECT).
- Every non-aggregated SELECT column must be in GROUP BY.
- WHERE filters rows; HAVING filters groups.
- NULL comparisons: use IS NULL, never = NULL.

A.6 Functional Dependencies

Armstrong's Axioms:

```
Reflexivity : Y ⊆ X => X -> Y
Augmentation : X -> Y => XZ -> YZ
Transitivity : X -> Y, Y -> Z => X -> Z
Union : X -> Y, X -> Z => X -> YZ
Decomposition : X -> YZ => X -> Y and X -> Z
```

Closure algorithm: start with X, keep adding RHS of FDs whose LHS is inside the current closure, until nothing changes.

Candidate key: attribute set X such that X^+ = all attrs AND no smaller subset has the same closure.

A.7 Normal Forms

Form	Rule
1NF	All attributes atomic (no multivalued, no composite)
2NF	1NF + no partial dependency of non-prime attr on candidate key
3NF	2NF + for every FD X -> A: X is superkey OR A is prime
BCNF	For every non-trivial FD X -> A: X is a superkey
4NF	BCNF + no non-trivial multi-valued dependencies

2NF matters only for composite PKs. Every BCNF is 3NF but not vice versa. BCNF is always lossless but may lose dependency preservation; **3NF is always both**.

A.8 Lossless Decomposition Test

```
R decomposed into R1 and R2 is lossless iff:
(R1 ∩ R2) -> R1 OR (R1 ∩ R2) -> R2
i.e., common attributes must be a superkey of at least one part.
```

A.9 ACID Properties

- **A**tomicity: all-or-nothing execution
- **C**onsistency: DB moves from valid state to valid state
- **I**solation: concurrent txns don't see partial changes
- **D**urability: committed changes survive crashes

A.10 Concurrency Control

Conflict: different txns, same data, at least one write.

Precedence graph: node per txn, edge Ti -> Tj on conflict. Schedule conflict-serializable iff graph is acyclic.

2PL phases:

- **Growing:** acquire locks only
- **Shrinking:** release locks only (no more acquires)

Strict 2PL: hold X-locks till commit (avoids cascade).

Rigorous 2PL: hold ALL locks till commit (used in practice).

Deadlock prevention:

- **Wait-Die**: older waits, younger dies (non-preemptive)
- **Wound-Wait**: older wounds younger, younger waits (preemptive)
- Both give priority to older transactions.

A.11 Indexing

- **Primary index**: on the file's ordering key. Sparse (one entry per block). One per table.
- **Secondary index**: on non-ordering attribute. Dense (one per record). Many per table.
- **Clustered index**: data physically ordered by key. Only one per table.
- **Non-clustered index**: index separate from data. Many per table. Extra lookup.
- **B+ Tree**: all data in leaves, leaves linked. Used by MySQL InnoDB, PostgreSQL, Oracle, SQL Server, and most others.
- **Search cost**: O(log_m N) where m is the fanout. With m ~= 100, billion-row table has depth ~5.

A.12 Common GATE Pitfalls

- SQL uses **bag semantics**; relational algebra uses **set semantics** (removes duplicates).
- **NULL** propagates through arithmetic and comparisons; use IS NULL / IS NOT NULL.
- **COUNT(*)** counts all rows; **COUNT(column)** ignores NULLs.
- **2NF** only applies when PK is composite.
- **BCNF decomposition** is always lossless but may not be dependency-preserving.
- **Cycle in precedence graph** means schedule is not conflict-serializable (but may still be view-serializable).
- **Conflict-serializable** => view-serializable; view-serializable does NOT imply conflict-serializable.

All the best for GATE & Placements — you've got this!